

Computer Vision

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

University of Bucharest, 2nd semester, 2025-2026

< All teams



Computer Vision 2025-2026 UniBuc/... ...

- Home page
- Class Notebook
- Classwork
- Assignments
- Grades
- Insights
- Reflect

- ▼ Main Channels
- General
 - Lab
 - Lecture

 **Lecture**

Posts

Shared ▼

Notes



 All documents

 In messages

 New ▼

 Upload ▼

 Edit in grid view

 Share

 Copy link

 Forms

New



Lecture

	Name ▼	Modified ▼	Modified By ▼
	Lecture01	February 23	DUMITRU BOGDAN A
	Lecture02	March 2	DUMITRU BOGDAN A
	Lecture03	6 days ago	DUMITRU BOGDAN A
	 CV-2025-2026-BONUS-KAHOOT-LECTURE....	A few seconds ago	DUMITRU BOGDAN A

Course structure

1. Low-level vision: Features and filters

Linear filters, color, texture, edge detection, template matching

2. Mid-level vision: Grouping and fitting

Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. Mid-level vision: Multiple views

Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. High-level vision: Object recognition

Object classification, object detection, part based models, bovw models

5. High-level vision: Video understanding

Object tracking, background subtraction, motion descriptors, optical flow

Course structure

1. Low-level vision: Features and filters

Linear filters, color, texture, edge detection, template matching

2. Mid-level vision: Grouping and fitting

Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. Mid-level vision: Multiple views

Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. High-level vision: Object recognition

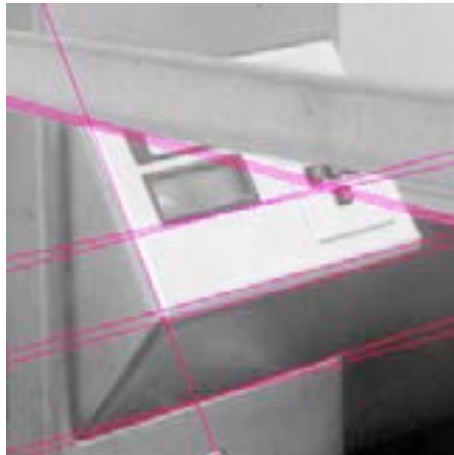
Object classification, object detection, part based models, bovw models

5. High-level vision: Video understanding

Object tracking, background subtraction, motion descriptors, optical flow

Fitting

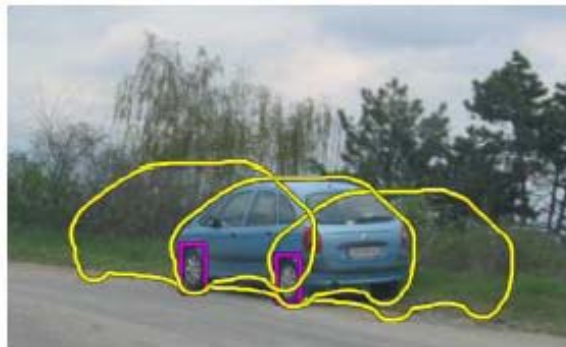
- Choose a *parametric model* to represent a set of features



simple model: lines



simple model: circles



complex model: car

Lab 2: Line detection - application

PROBĂ DE CONCURS

Domeniul... CTI **730**

Sesiunea... IULIE 2018

Nota pe lucrare 4,90 (până 5,5) Nota după contestație



TEST GRILĂ

MATEMATICĂ

Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

INFORMATICĂ ☐

FIZICĂ ☒

Număr întrebare	Răspuns			
	A	B	C	D
1	X			
2			X	
3		X		
4	X			
5		X		
6	X			
7	X			
8		X		
9				X
10			X	
11		X		
12				X
13			X	
14	X			
15			X	

NOTĂ : Se bifează X în căsuța corespunzătoare răspunsului corect.

Lab 2: Line detection - application

MATEMATICA

Număr întrebare	Răspuns			
	A	B	C	D
1	X			
2				X
3		X		
4			X	
5	X			
6			X	
7	X			
8			X	
9		X		
10	X			
11				X
12			X	
13			X	
14		X		
15				X

Edge image for query

MATEMATICA

Număr întrebare	Răspuns			
	A	B	C	D
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				

NOTĂ: Se bifează X în căuțele corespunzătoare

Edge image for template

Lab 2: Line detection - application

Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

Lab 2: Line detection - application

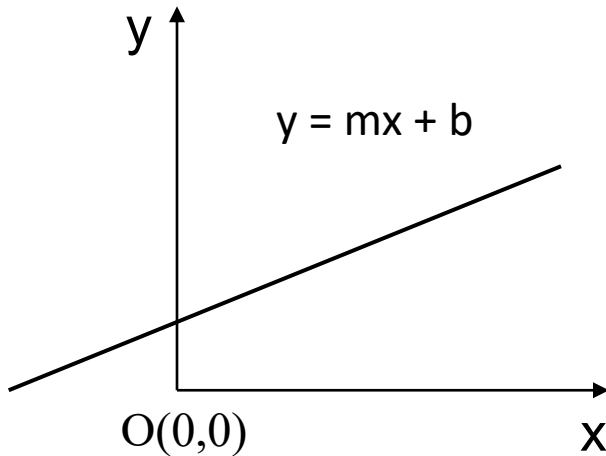
MATEMATICA

Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

MATEMATICA

Număr întrebare	Răspuns			
	A	B	C	D
1			X	
2		X		
3			X	
4	X			
5			X	
6			X	
7		X		
8				X
9				X
10	X			
11		X		
12		X		
13			X	
14				X
15				X

Parameterization of a line



Parameterization of a line in image plane: $y = m * x + b$,

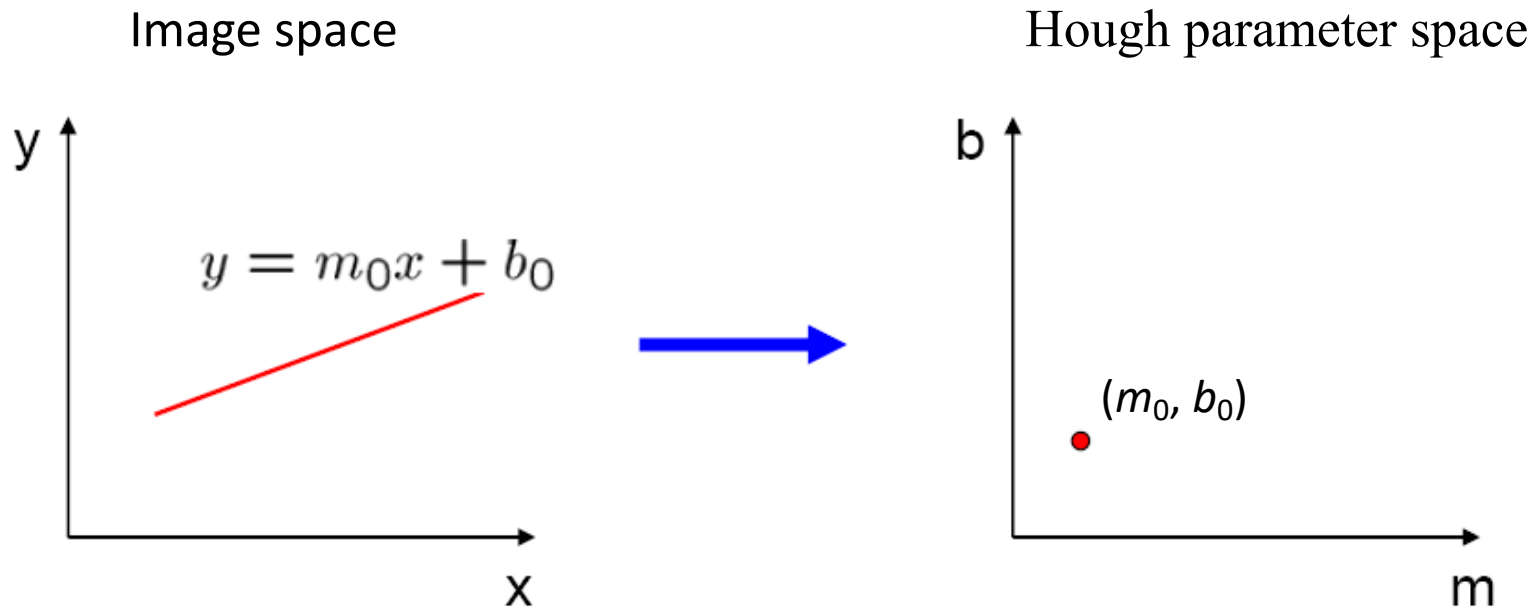
m – slope (defines the angle between the line and Ox axis),

b – intercept (bias to origin $O(0,0)$)

(m,b) are line parameters

Parameter space representation

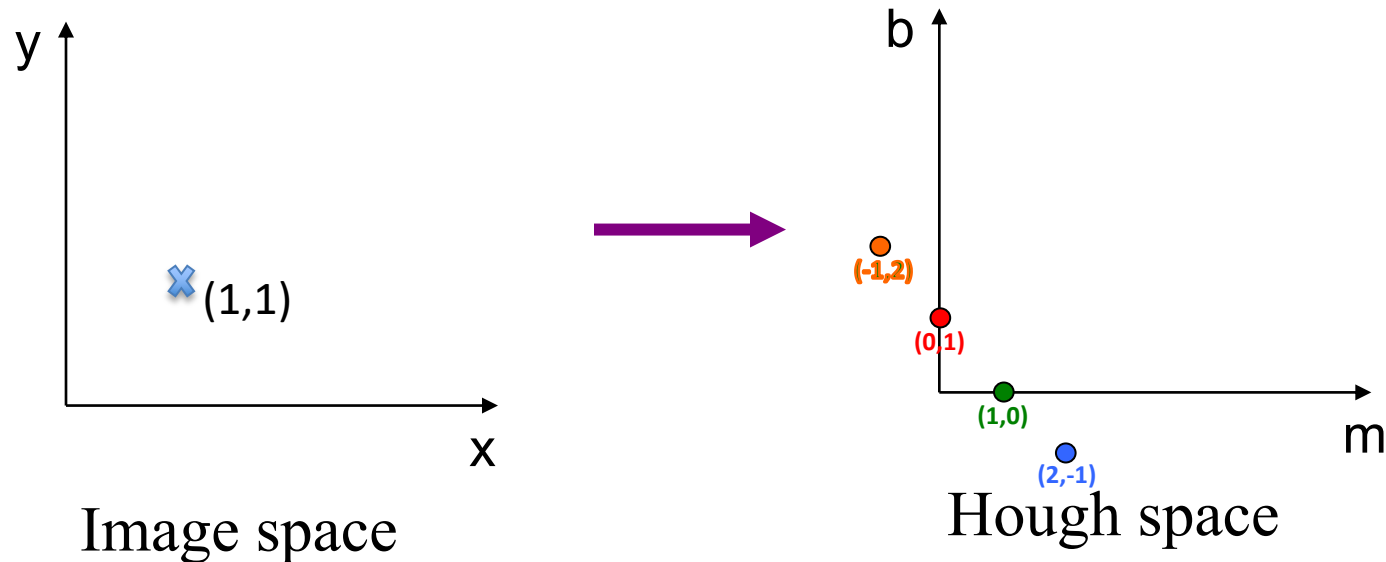
- A **line** in the image corresponds to a **point** in Hough space



Correspondence between image and Hough spaces

A line in the image space corresponds to a point in the Hough space.

A **point** (x_0, y_0) in the image space maps to a line in the Hough space



There is an infinity of lines of the form $y = mx + b$ that contain point $(1,1)$:

$y = x$ ($m=1, b=0$); $y = 2x-1$ ($m=2, b=-1$), $y = 1$ ($m=0, b = 1$), $y = -x + 2$ ($m=-1, b = 2$),

All 4 points $(1,0)$, $(2,-1)$, $(0,-1)$, $(-1,2)$ are on the line $b = -m+1$

Correspondence between image and Hough spaces

A line in the image space corresponds to a point in the Hough space.

A **point** (x_0, y_0) in the image space maps to a line in the Hough space

Parameterization of a line in image plane: $y = m * x + b$,

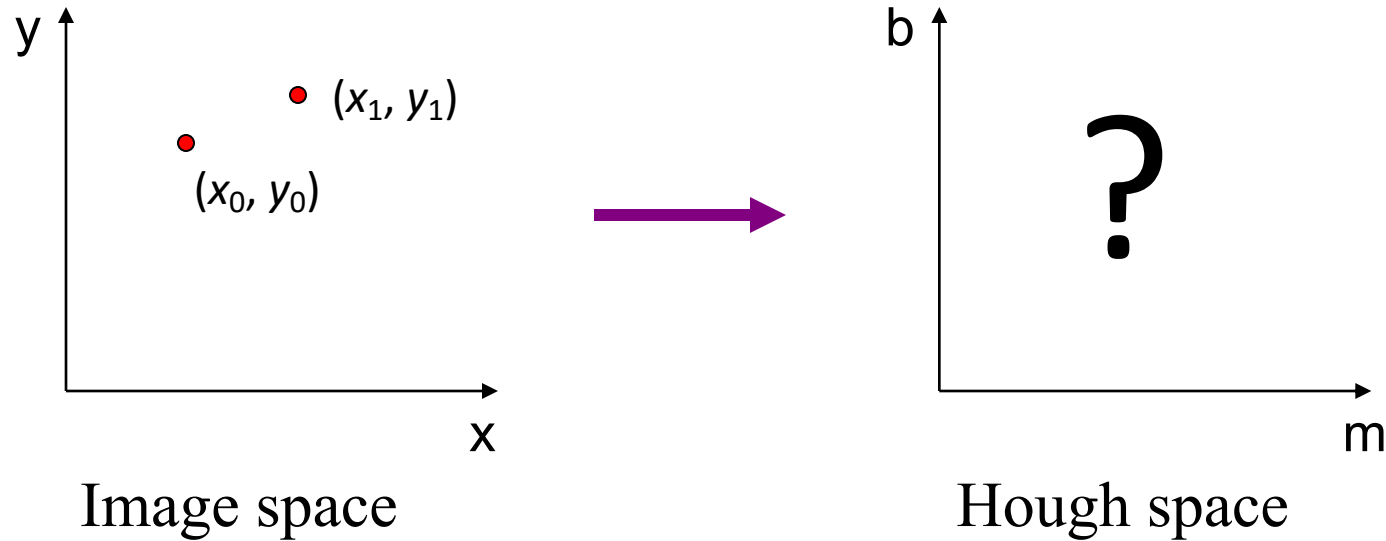
All lines that contain point (x_0, y_0) can have any slope m and any intercept b , the only condition that should be satisfied is

$$y_0 = m * x_0 + b.$$

$y_0 = m * x_0 + b \Leftrightarrow b = -x_0 * m + y_0$ (parameterization of the line in the Hough space with slope $-x_0$ and intercept y_0)

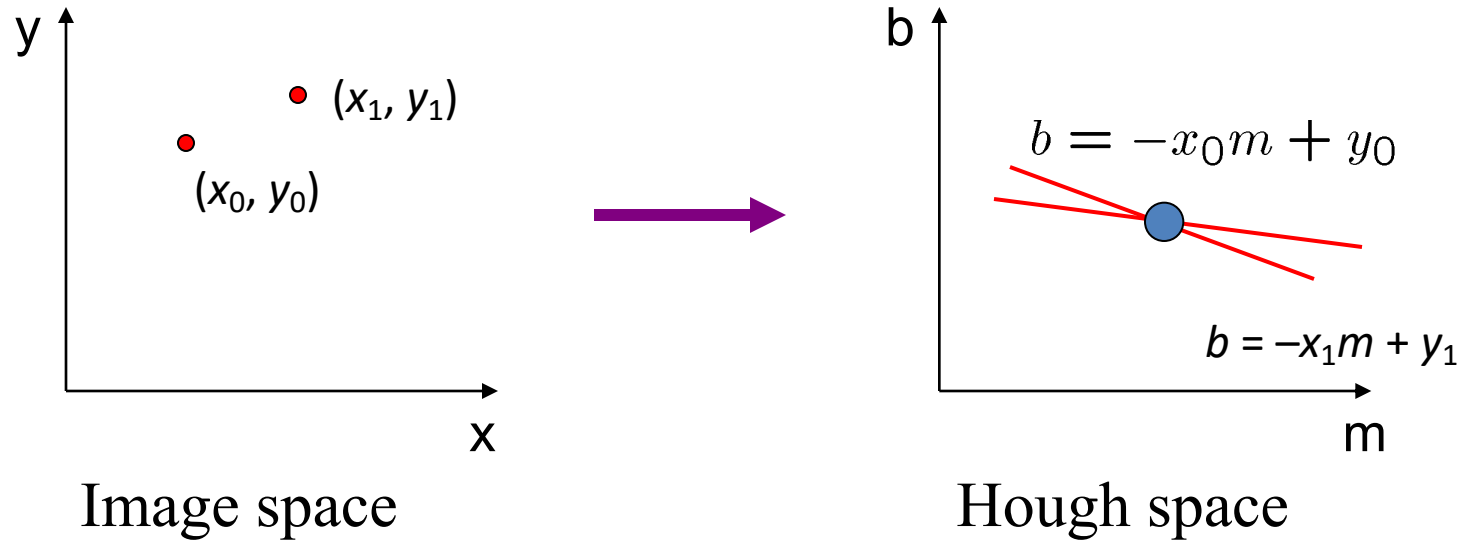
$$(x_0, y_0) = (1, 1) \Rightarrow b = -m + 1$$

Parameter space representation



Where is the line that contains both (x_0, y_0) and (x_1, y_1) ?

Parameter space representation



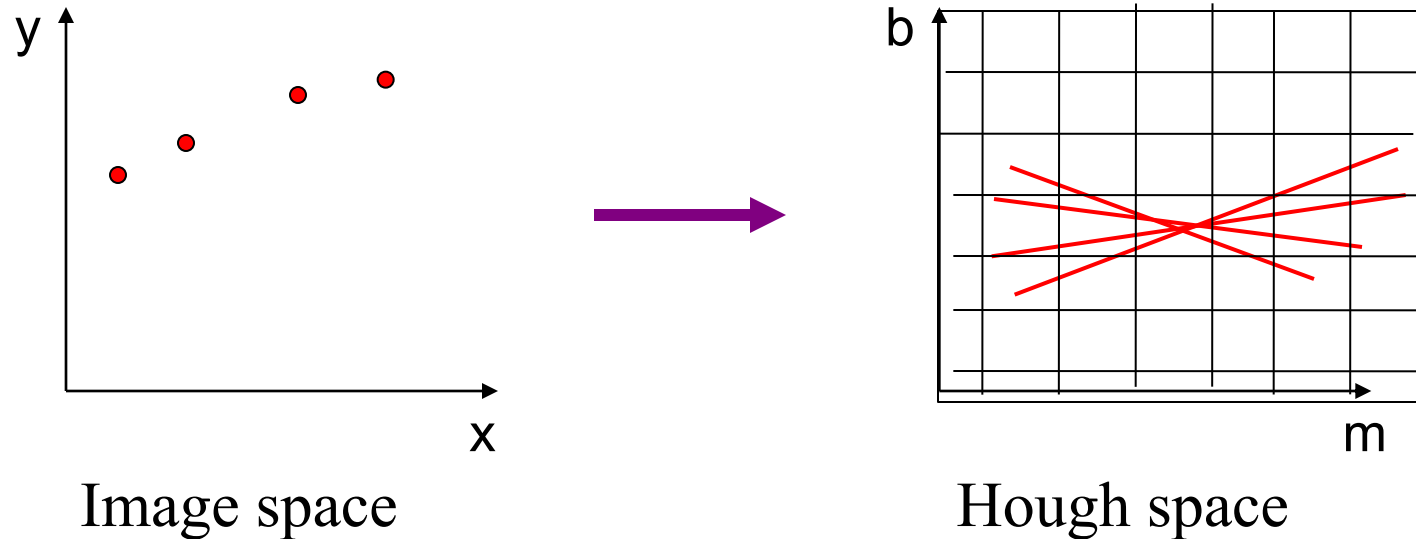
Where is the line that contains both (x_0, y_0) and (x_1, y_1) ?

Point (x_0, y_0) maps to the line in the Hough space: $b = -x_0m + y_0$

Point (x_1, y_1) maps to the line in the Hough space: $b = -x_1m + y_1$

The line that contains both (x_0, y_0) and (x_1, y_1) maps to a point in the Hough space. This point is the intersection of lines $b = -x_0m + y_0$ and $b = -x_1m + y_1$

Finding lines in an image with Hough



How we can use the previous observations to find parameters (m, b) that define the most probable line in the image space?

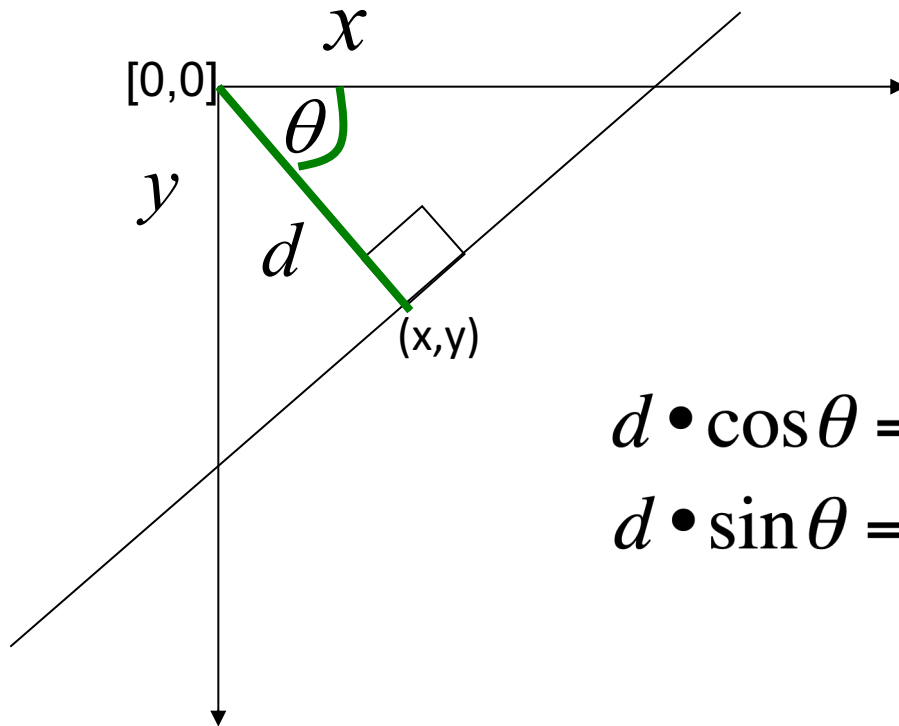
- each edgel point (part of an edge) from image space will vote (for an infinity number of lines) in the Hough space (here a line is a point)
- discretize the Hough space in discrete intervals/cells; the cell with largest number of votes determines the parameters of the line

Polar representation

Problems with the (m,b) space:

- unbounded parameter domains
- vertical lines require infinite m

Alternative: **polar representation**



d : distance from origin to line
(length of the perpendicular)

θ : angle between the
perpendicular and Ox axis

$$d \cdot \cos \theta = x$$

$$d \cdot \sin \theta = y$$

$$d \cdot (\cos \theta)^2 = x \cdot \cos \theta$$

$$d \cdot (\sin \theta)^2 = y \cdot \sin \theta$$

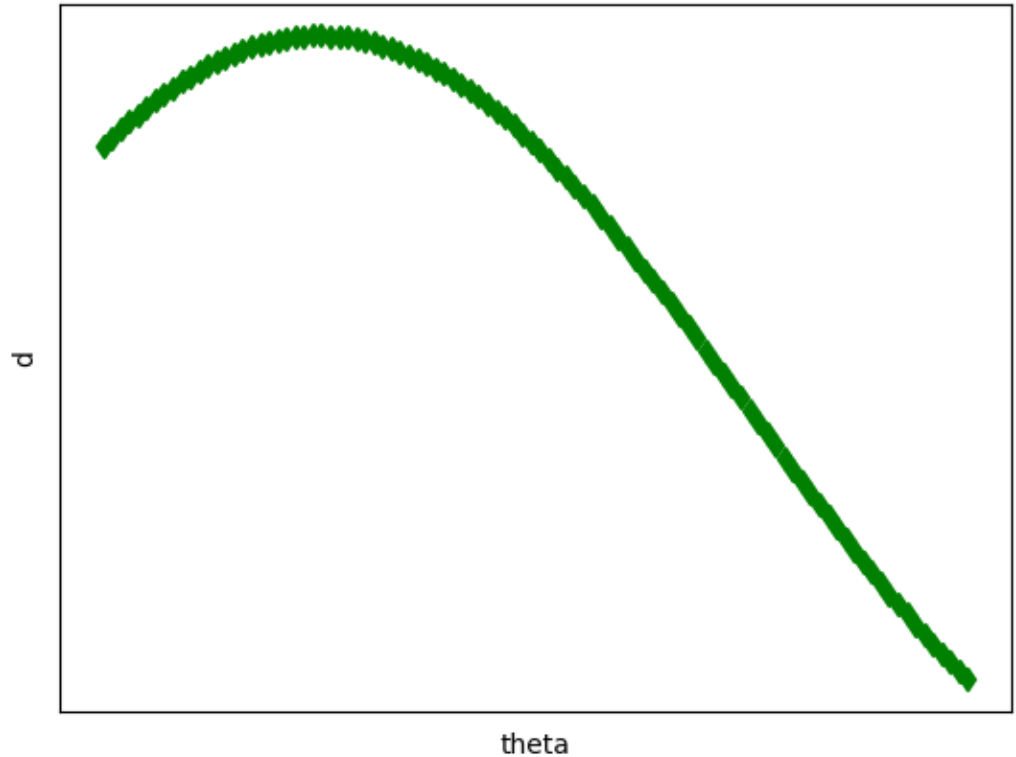
$$d = x \cdot \cos \theta + y \cdot \sin \theta$$

Point in the image space $\rightarrow$ sinusoid curve in the Hough space

plot_sinusioid_curve.py x

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x,y = 1,1
4 theta = np.linspace(0,np.pi,100)
5 d = x * np.cos(theta) + y*np.sin(theta)
6 plt.figure
7 plt.plot(theta,d,"dg")
8 plt.xlabel('theta')
9 plt.ylabel('d')
10 plt.title('Plot of the function d = cos(theta) + sin(theta)')
11 plt.xticks([])
12 plt.yticks([])
13 plt.show()
```

Plot of the function $d = \cos(\theta) + \sin(\theta)$

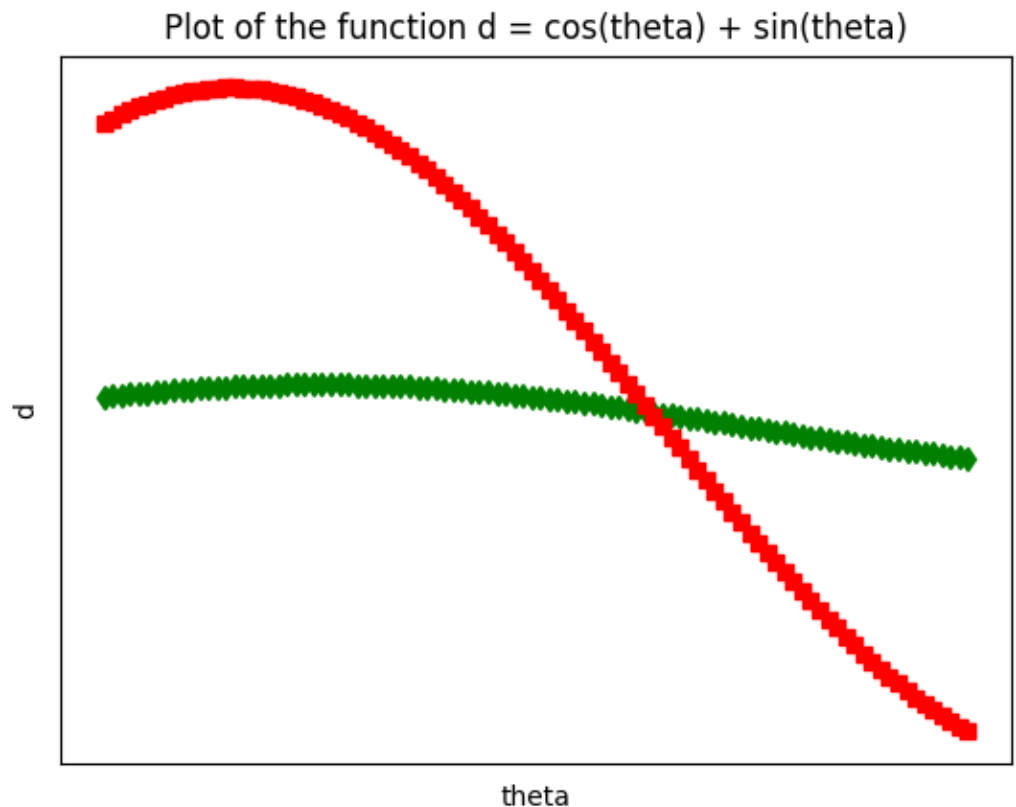


```
plot_sinusioid_curve.py x
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x,y = 1,1
4 theta = np.linspace(0,np.pi,100)
5 d = x * np.cos(theta) + y*np.sin(theta)
6 plt.figure
7 plt.plot(theta,d,"dg")
8 plt.xlabel('theta')
9 plt.ylabel('d')
10 plt.title('Plot of the function d = cos(theta) + sin(theta)')
11 plt.xticks([])
12 plt.yticks([])
13 #plt.show()
14
15
16 x,y = 10,5
17 d1 = x * np.cos(theta) + y*np.sin(theta)
18 plt.plot(theta,d1,"sr")
19 plt.show()
```

Any two points define a unique line.

⇒ Therefore, the corresponding sinusoidal curves in Hough space intersect.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x,y = 1,1
4 theta = np.linspace(0,np.pi,100)
5 d = x * np.cos(theta) + y*np.sin(theta)
6 plt.figure
7 plt.plot(theta,d,"dg")
8 plt.xlabel('theta')
9 plt.ylabel('d')
10 plt.title('Plot of the function d = cos(theta) + sin(theta)')
11 plt.xticks([])
12 plt.yticks([])
13 #plt.show()
14
15
16 x,y = 10,5
17 d1 = x * np.cos(theta) + y*np
18 plt.plot(theta,d1,"sr")
19 plt.show()
```



Hough transform-algorithm outline

- Initialize accumulator H to all zeros
- For each feature point (x,y) in the image

For $\theta = 0$ to 180

$$d = x \cos \theta + y \sin \theta$$

$$H(\theta, d) = H(\theta, d) + 1$$

end

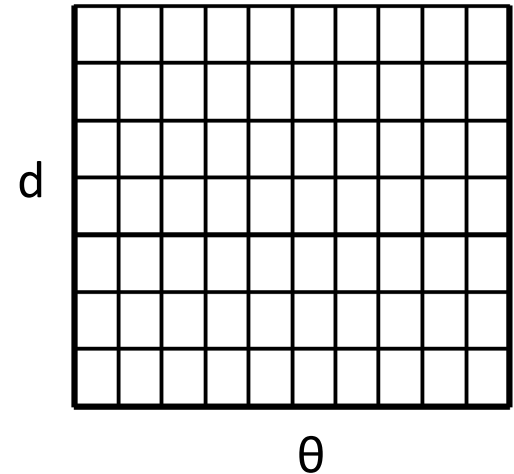
end

- Find the value(s) of (θ, d) where $H(\theta, d)$ is a local maximum

– The detected line in the image is given by

$$d = x \cos \theta + y \sin \theta$$

H: accumulator array (votes)

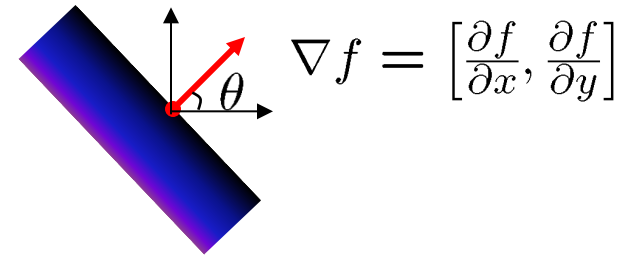


Hough transform - demo

<https://www.aber.ac.uk/~dcswww/Dept/Teaching/CourseNotes/current/CS34110/hough.html>

Incorporating image gradients

- When we detect an edge point, we also know its gradient orientation
- How does this constrain possible lines passing through the point?



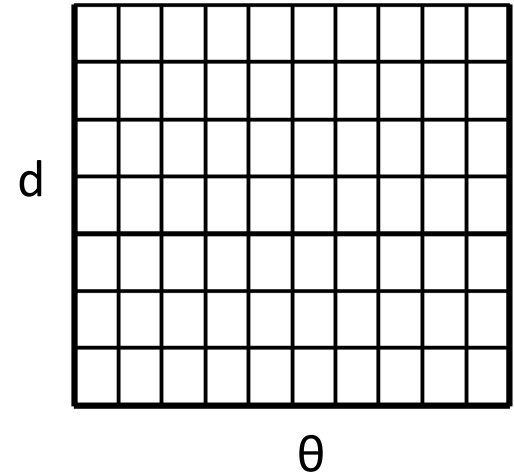
$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

- Modified Hough transform
- For each edge point (x,y)
 - $\theta = \text{gradient orientation at (x,y)}$
 - $d = x \cos \theta + y \sin \theta$
 - $H(\theta, d) = H(\theta, d) + 1$
- end

Algorithm outline with image gradients

- Initialize accumulator H to all zeros
 - For each feature point (x,y) in the image
 - $\theta = \text{gradient orientation at } (x,y)$
 - $d = x \cos \theta + y \sin \theta$
 - $H(\theta, d) = H(\theta, d) + 1$
- end

H: accumulator array (votes)



- Find the value(s) of (θ, d) where $H(\theta, d)$ is a local maximum
 - The detected line in the image is given by
 - $d = x \cos \theta + y \sin \theta$

Algorithm outline with image gradients

- Initialize accumulator H to all zeros
- For each feature point (x,y) in the image

$\theta = \text{gradient orientation at (x,y)}$

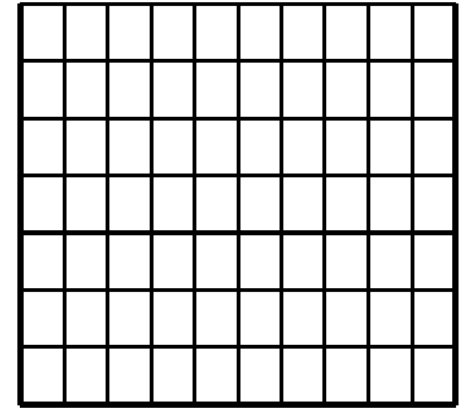
$\text{mag} = \text{gradient magnitude at (x,y)}$

$d = x \cos \theta + y \sin \theta$

$H(\theta, d) = H(\theta, d) + \text{mag}$

end

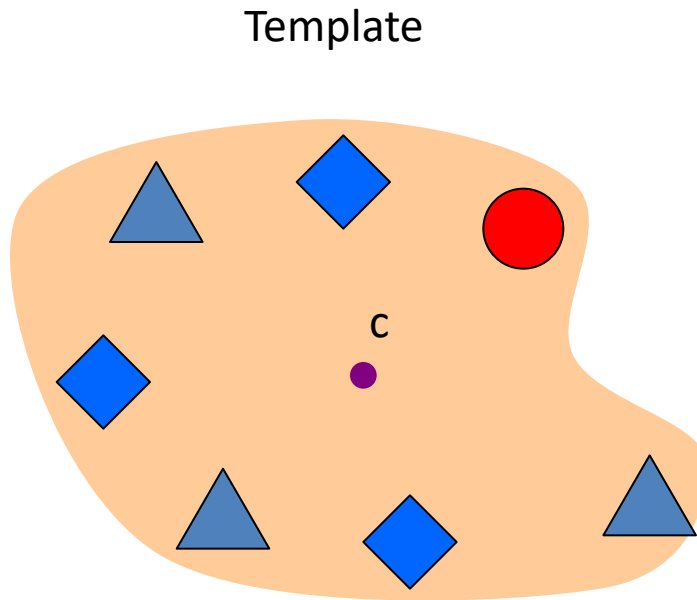
H: accumulator array (votes)



- Find the value(s) of (θ, d) where $H(\theta, d)$ is a local maximum
 - The detected line in the image is given by
$$d = x \cos \theta + y \sin \theta$$

Generalized Hough transform

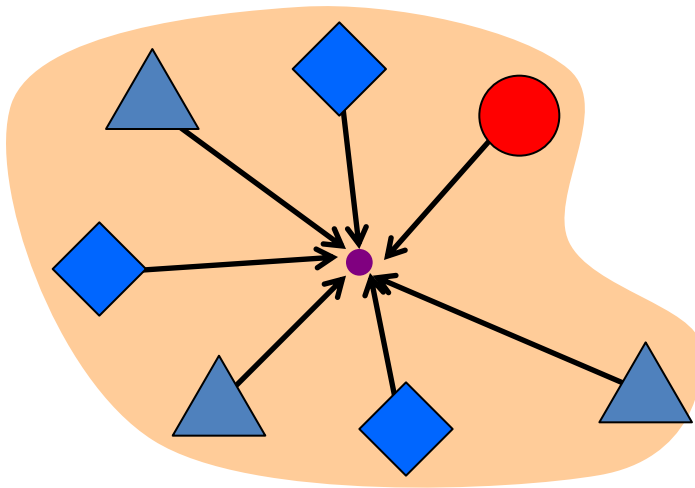
- We want to find a template defined by its reference point (center) and several distinct types of landmark points in stable spatial configuration



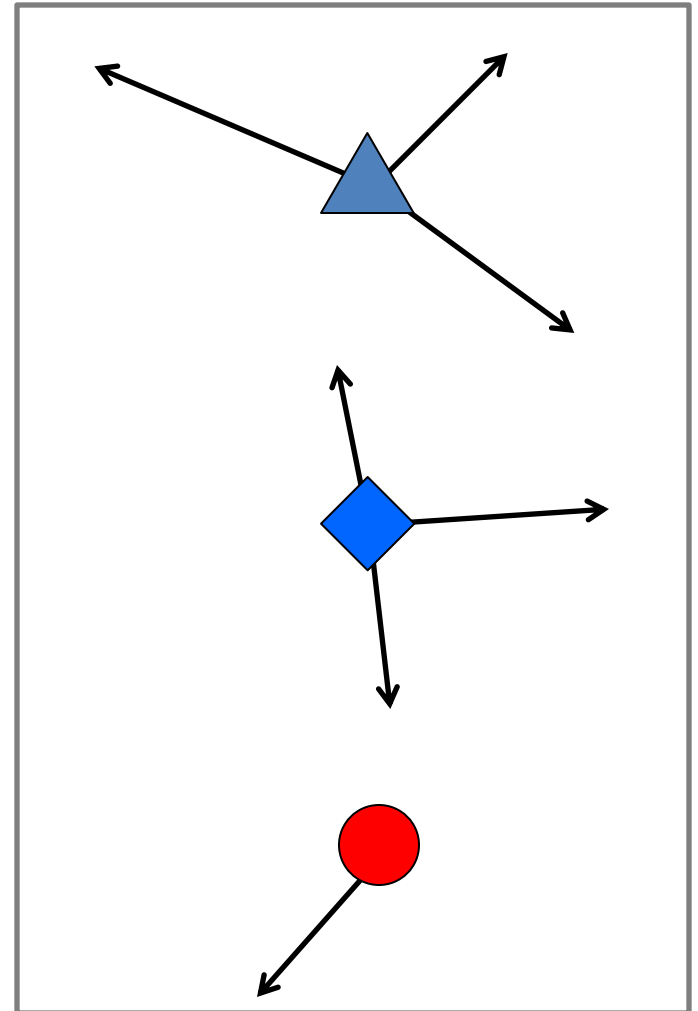
Generalized Hough transform

- Template representation: for each type of landmark point, store all possible displacement vectors towards the center

Template

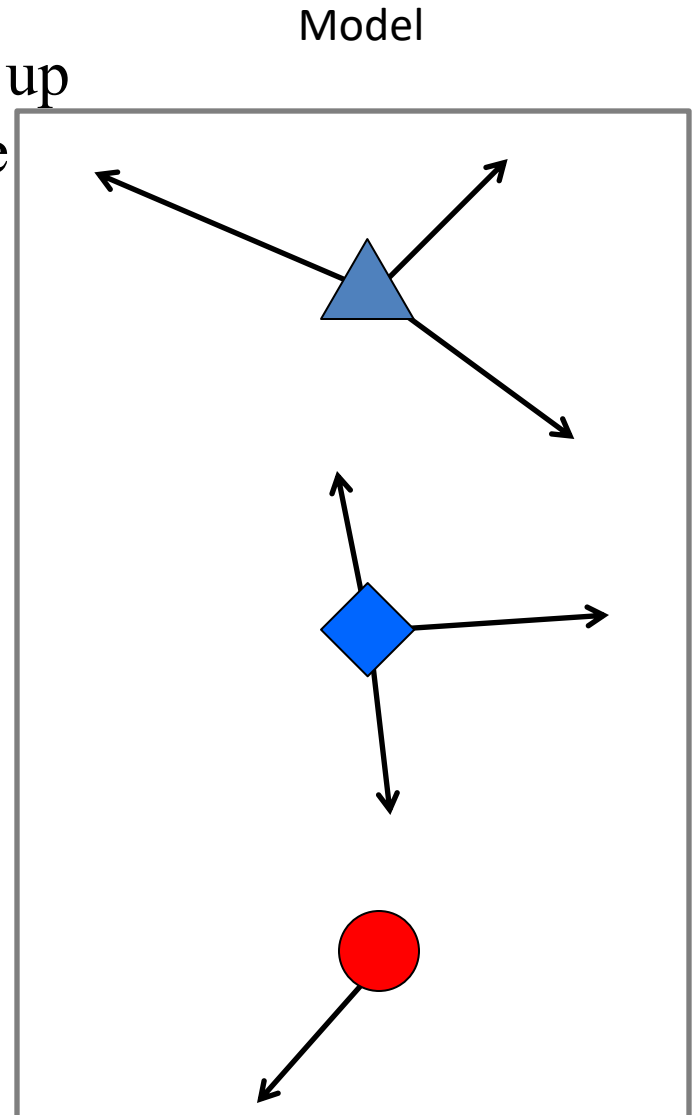
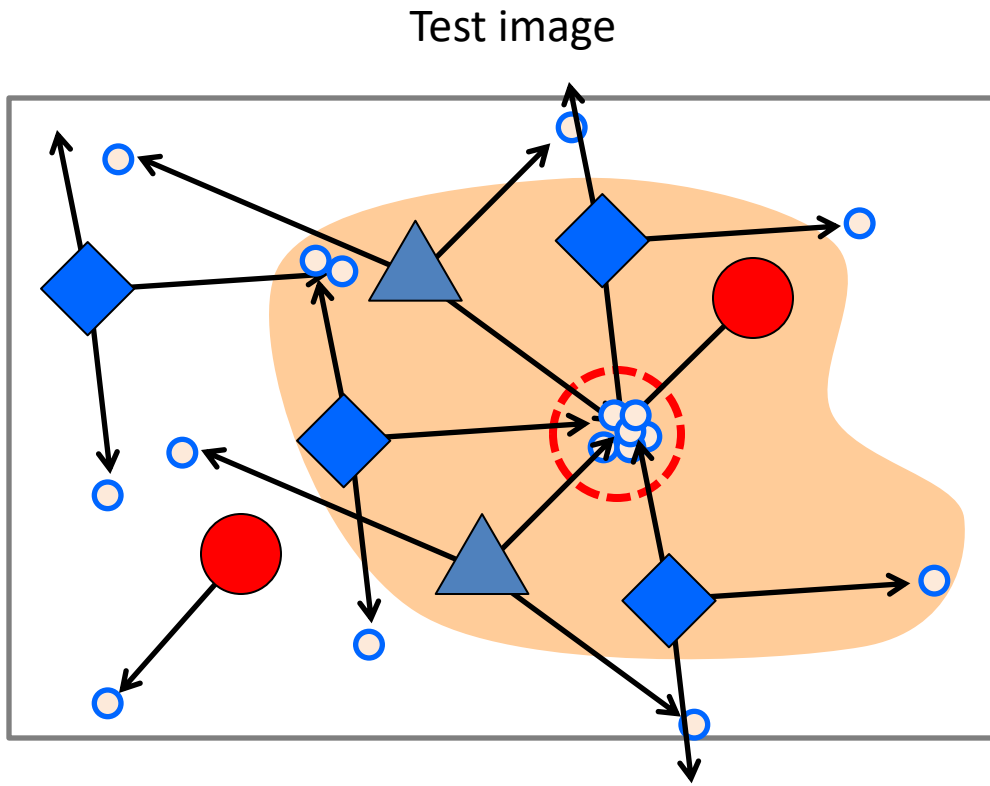


Model



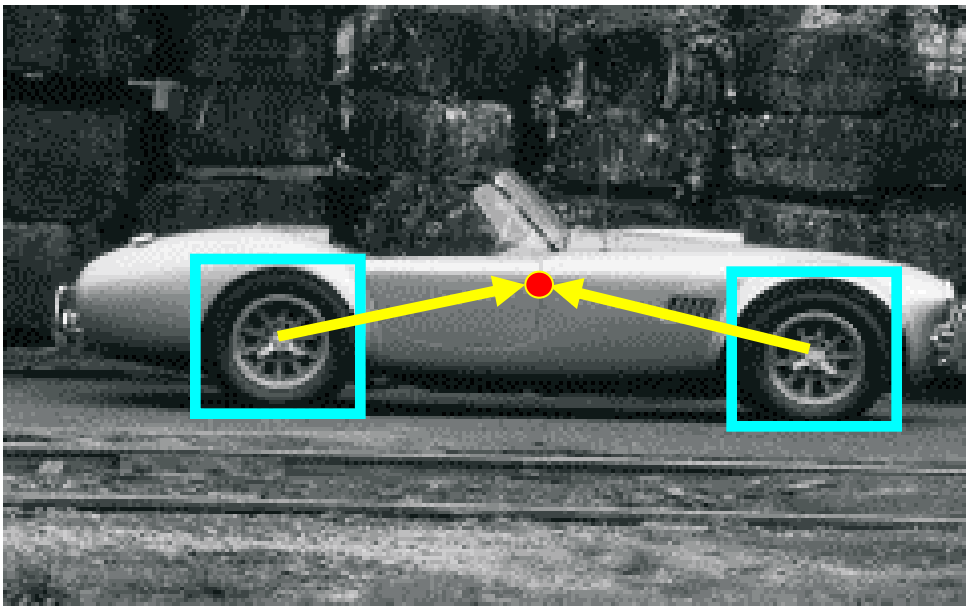
Generalized Hough transform

- Detecting the template:
 - For each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model

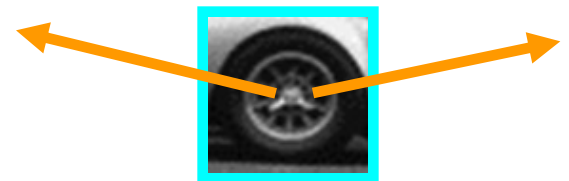


Application in recognition

- Index displacements by “visual codeword”



training image



visual codeword with
displacement vectors

B. Leibe, A. Leonardis, and B. Schiele, [Combined Object Categorization and Segmentation with an Implicit Shape Model](#), ECCV Workshop on Statistical Learning in Computer Vision 2004

Application in recognition

- Index displacements by “visual codeword”

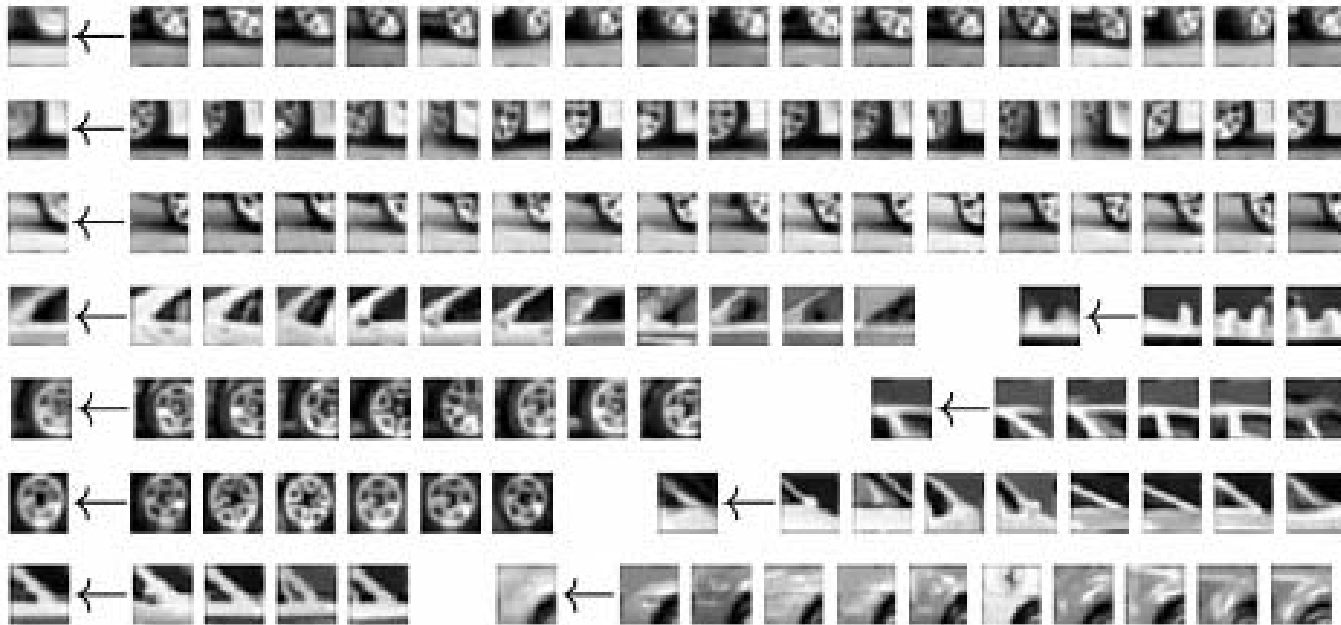


test image

B. Leibe, A. Leonardis, and B. Schiele, [Combined Object Categorization and Segmentation with an Implicit Shape Model](#), ECCV Workshop on Statistical Learning in Computer Vision 2004

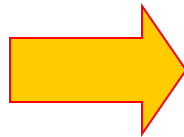
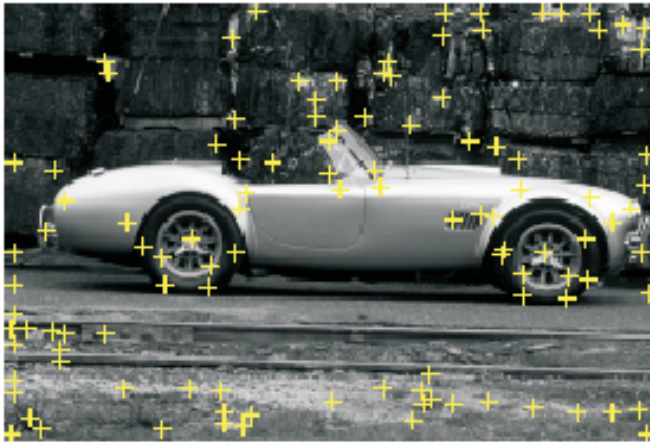
Implicit shape models: Training

1. Build *codebook* of patches around extracted interest points using clustering



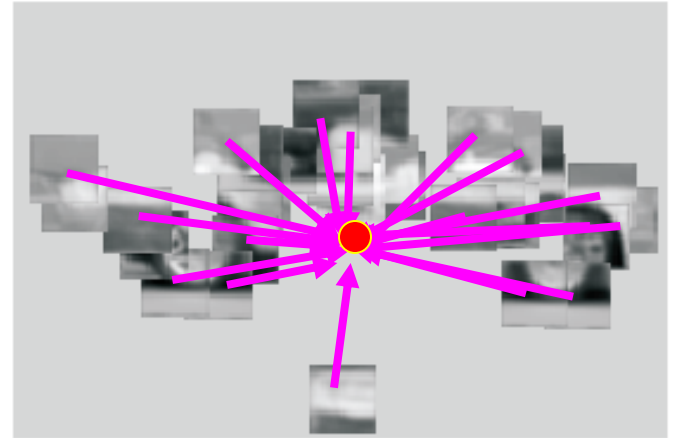
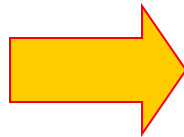
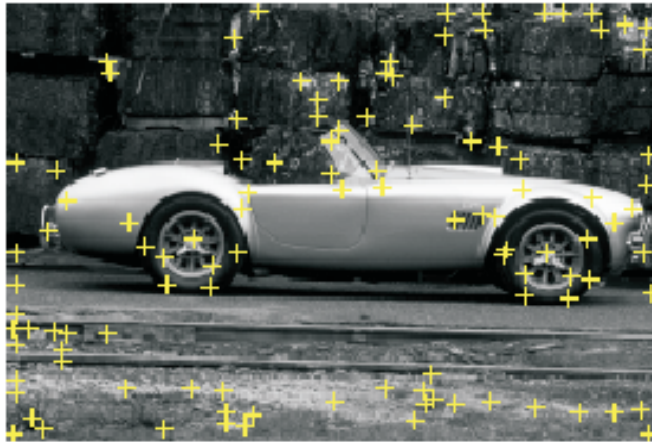
Implicit shape models: Training

1. Build *codebook* of patches around extracted interest points using clustering
2. Map the patch around each interest point to closest codebook entry



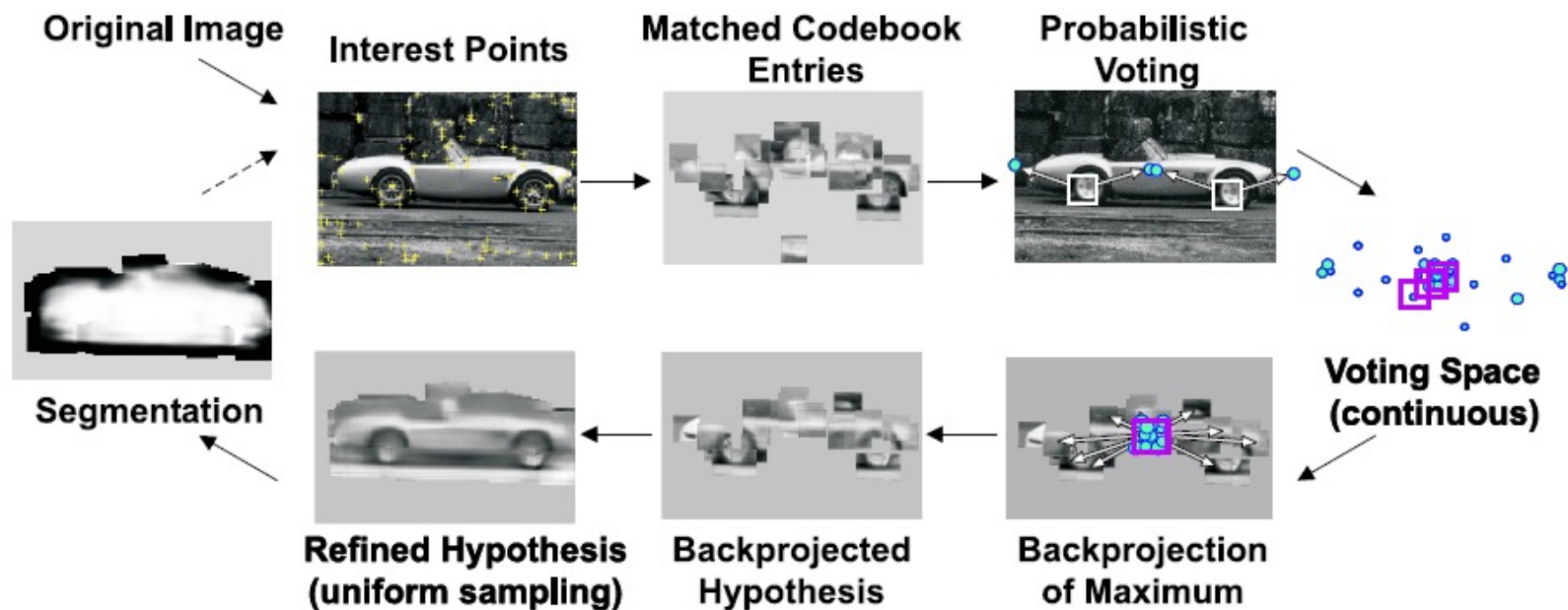
Implicit shape models: Training

1. Build *codebook* of patches around extracted interest points using clustering
2. Map the patch around each interest point to closest codebook entry
3. For each codebook entry, store all positions it was found, relative to object center

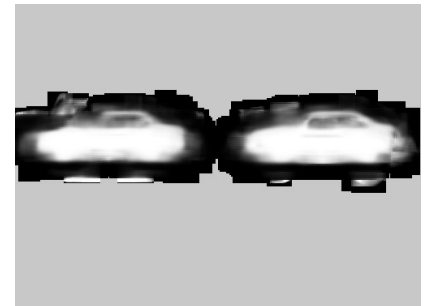
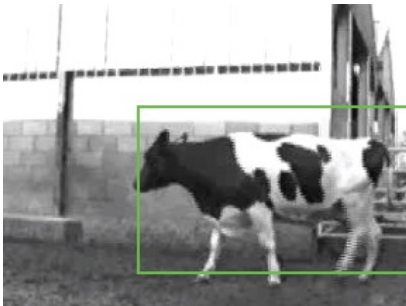
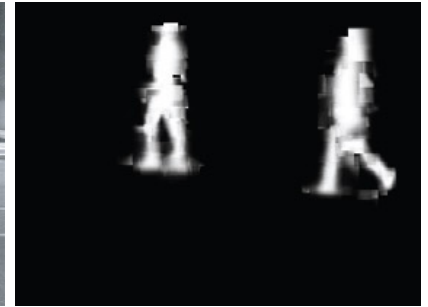
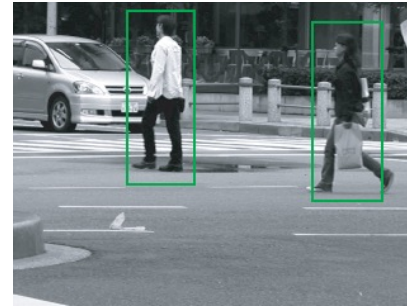
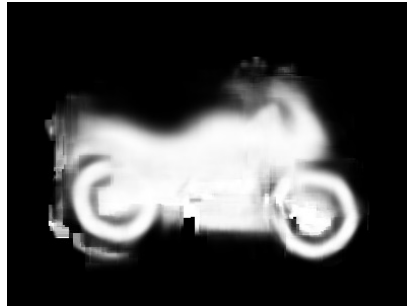


Implicit shape models: Testing

1. Given test image, extract patches, match to codebook entry
2. Cast votes for possible positions of object center
3. Search for maxima in voting space
4. Extract weighted segmentation mask based on stored masks for the codebook occurrences



Additional examples



B. Leibe, A. Leonardis, and B. Schiele, [Robust Object Detection with Interleaved Categorization and Segmentation](#),
IJCV 77 (1-3), pp. 259-289, 2008.

From Implicit Shape Models to Modern Instance Segmentation

Implicit Shape Model

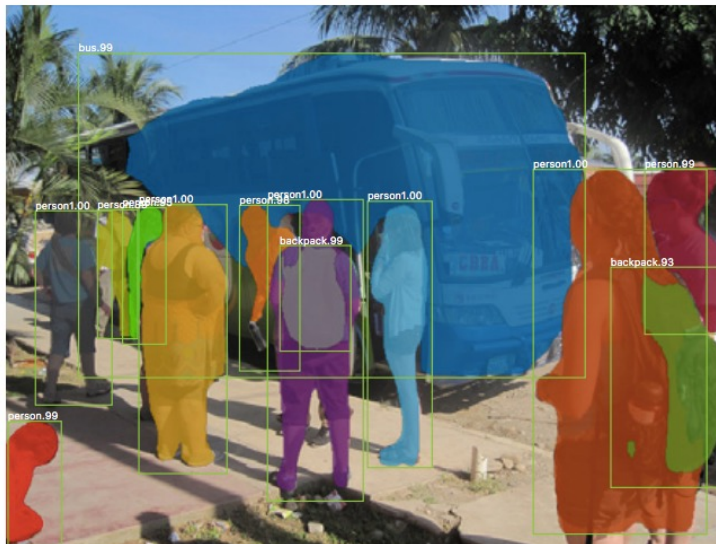
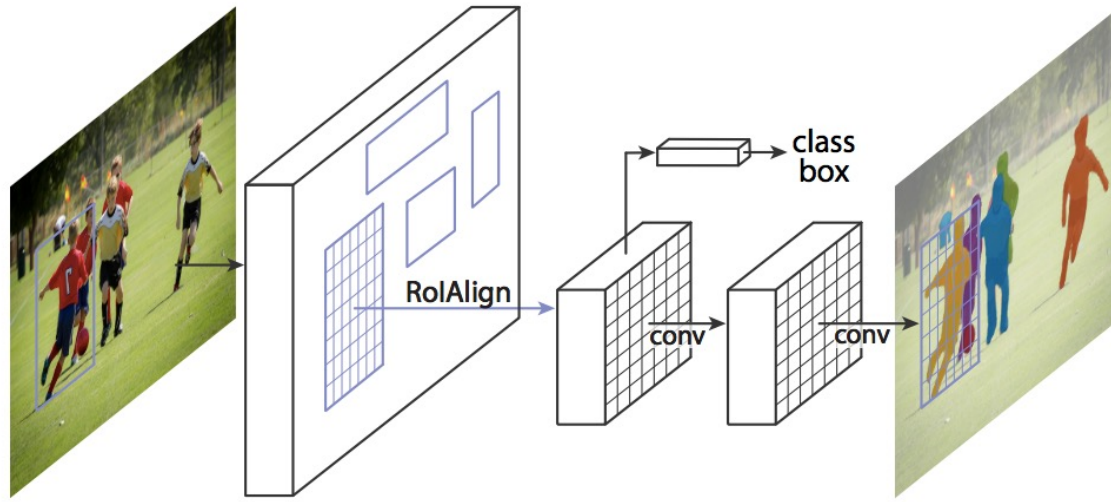
- Hand-crafted features (SIFT)
- Codebook of visual words
- Hough voting for object center
- Segmentation from stored masks

Modern approaches (Mask R-CNN)

- Deep convolutional features
- Region proposals learned automatically
- End-to-end object detection
- Direct pixel-wise mask prediction

Implicit Shape Models were among the first methods that attempted to detect objects and recover their segmentation simultaneously. However, they relied on hand-crafted features and a voting scheme. Modern approaches such as Mask R-CNN learn both detection and segmentation directly from data using deep neural networks.

These days: Mask R-CNN

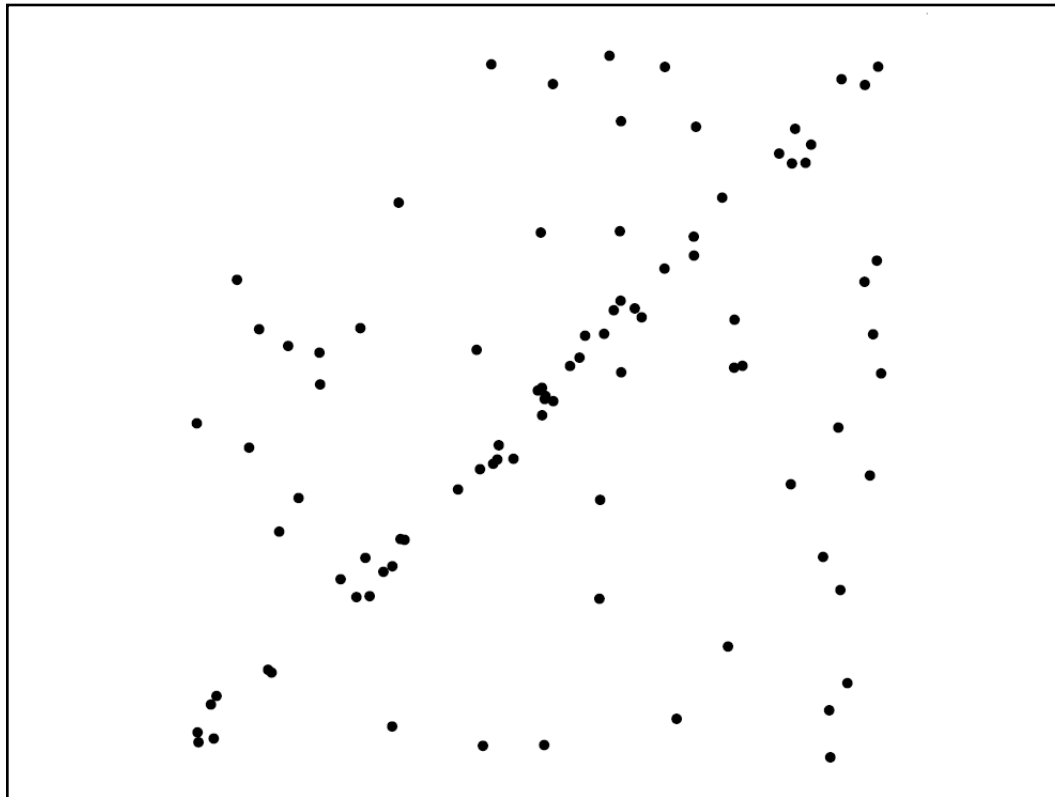


RANSAC

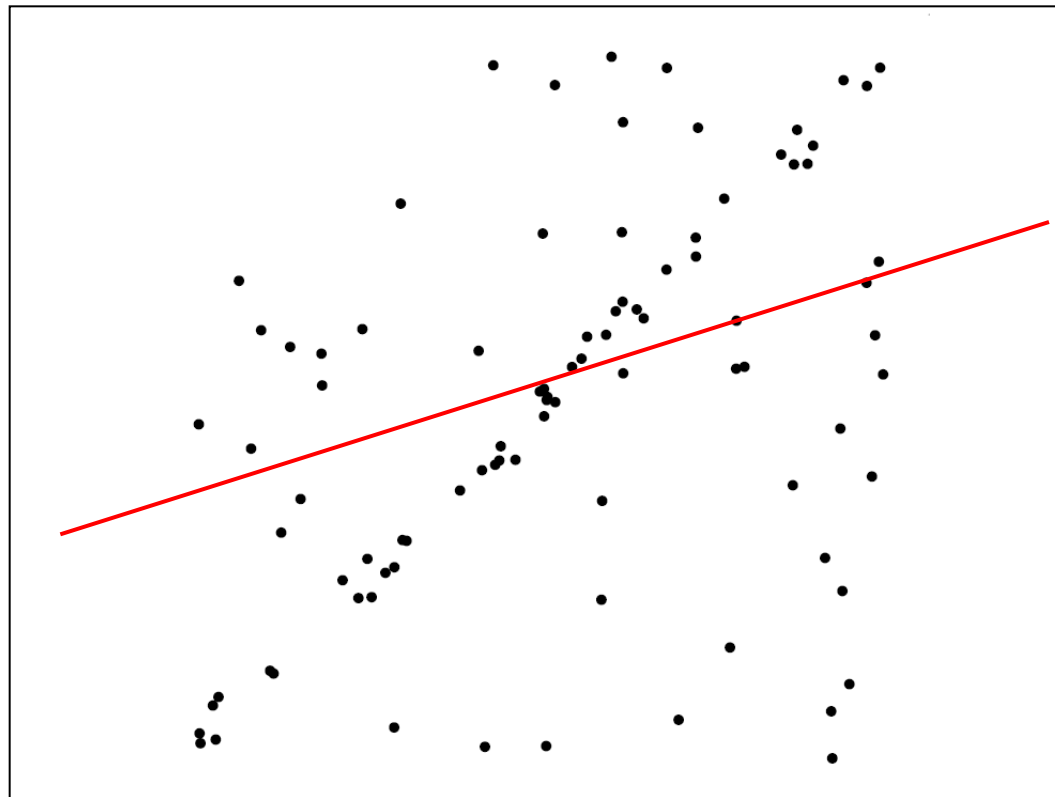
RANSAC

- Random sample consensus (RANSAC):
Very general framework for model fitting in the presence of outliers
- Outline
 - Choose a small subset of points uniformly at random
 - Fit a model to that subset
 - Find all remaining points that are “close” to the model and reject the rest as outliers
 - Do this many times and choose the best model

RANSAC for line fitting example

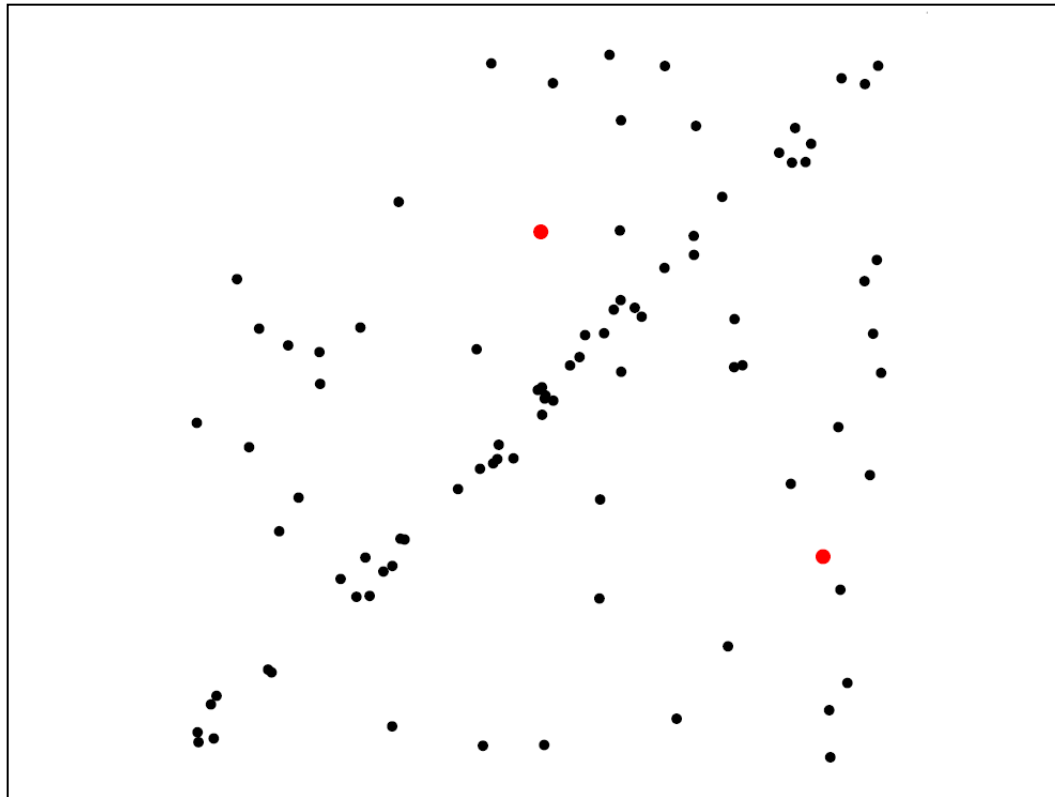


RANSAC for line fitting example



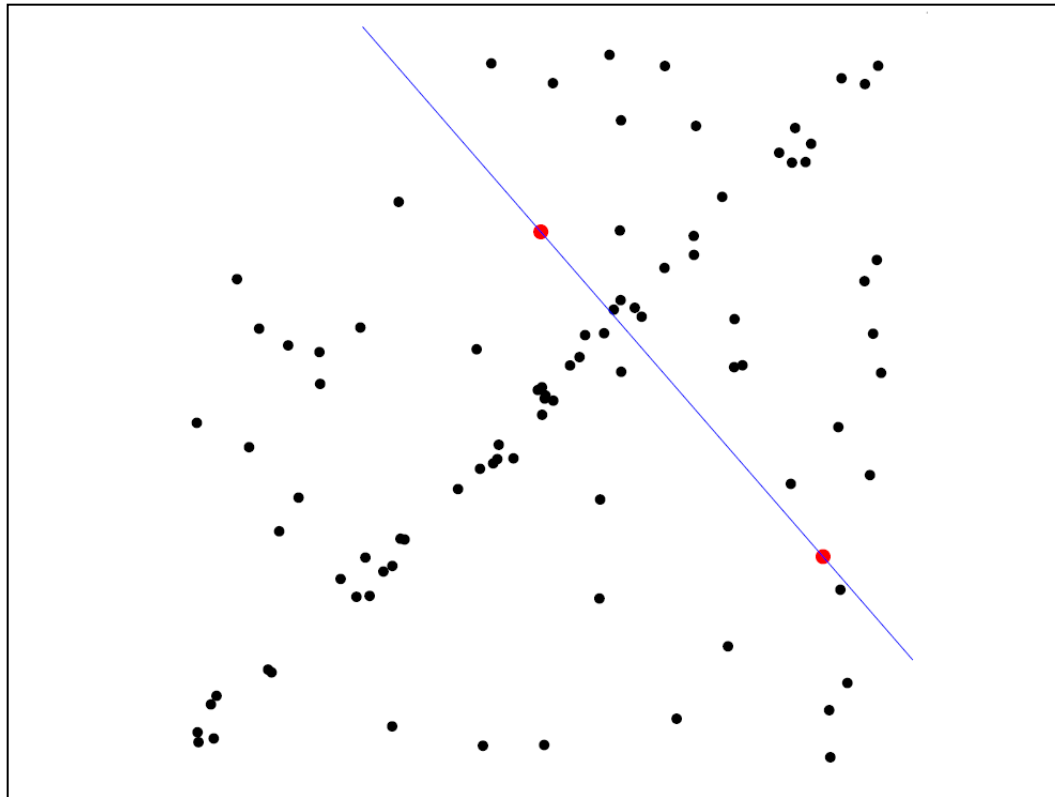
Least-squares fit

RANSAC for line fitting example



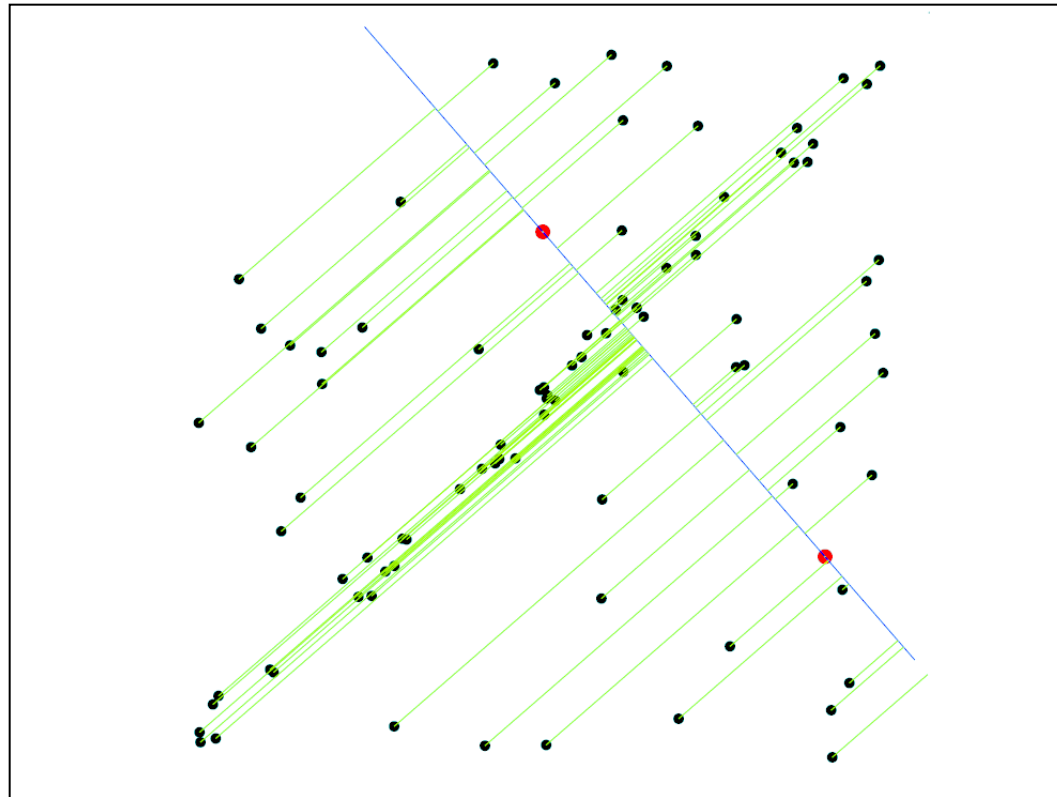
1. Randomly select minimal subset of points

RANSAC for line fitting example



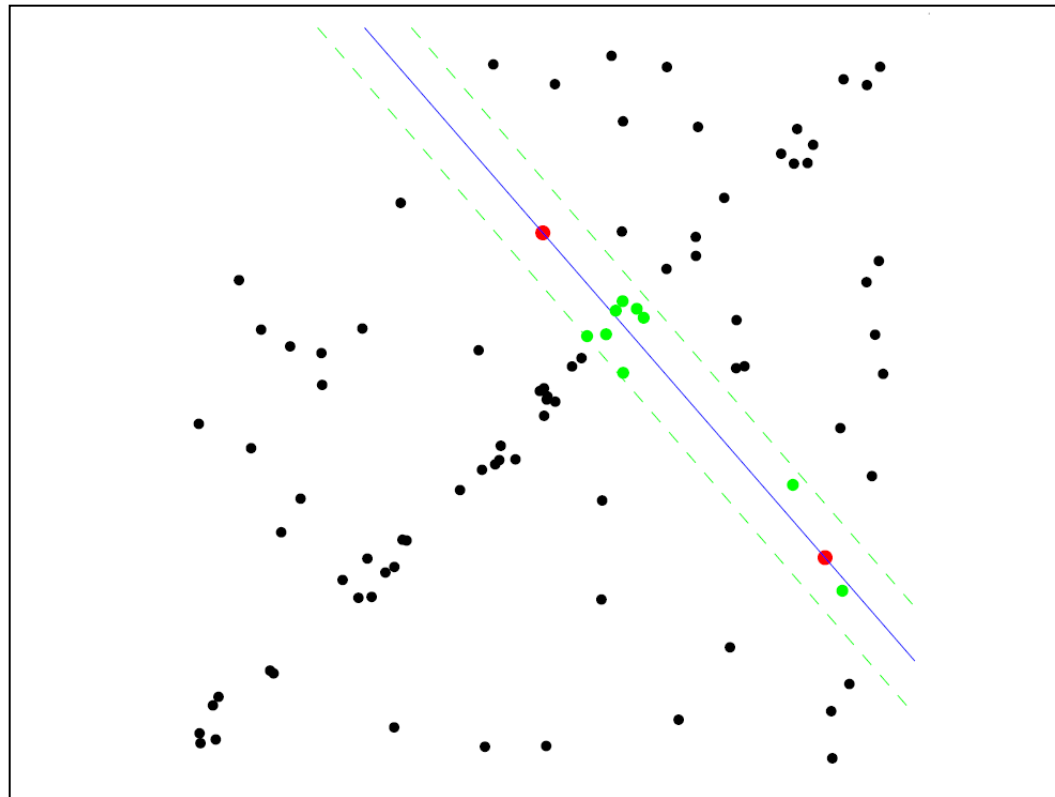
1. Randomly select minimal subset of points
2. Hypothesize a model

RANSAC for line fitting example



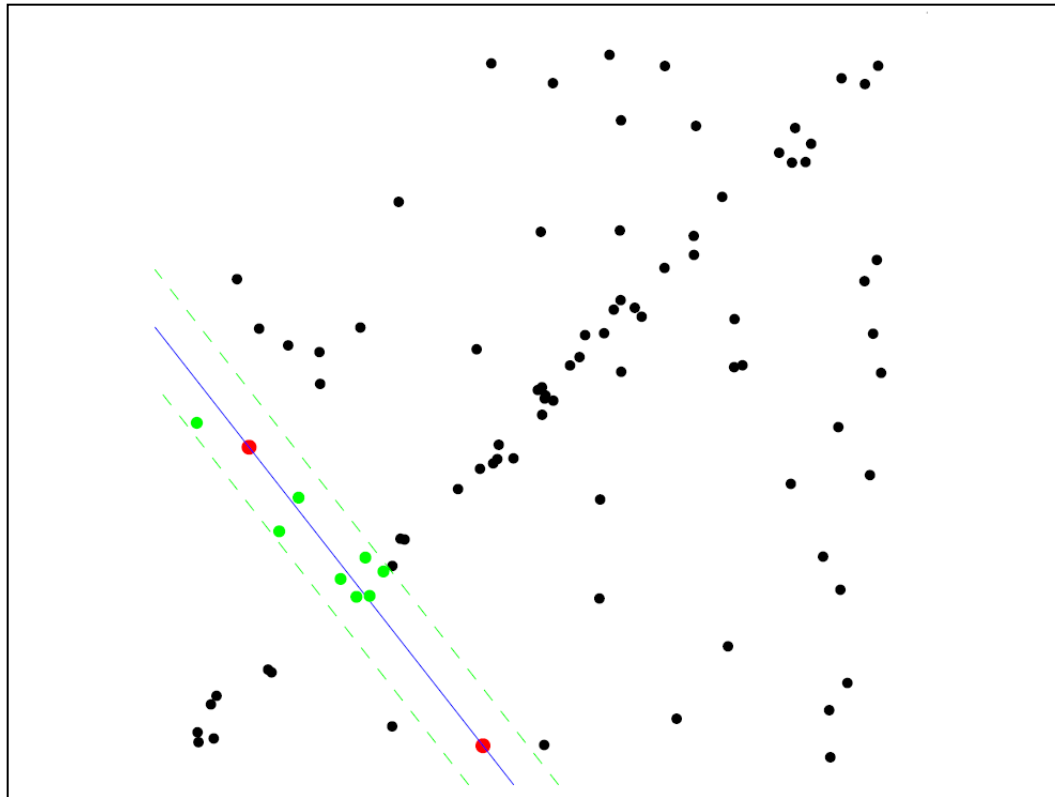
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function

RANSAC for line fitting example



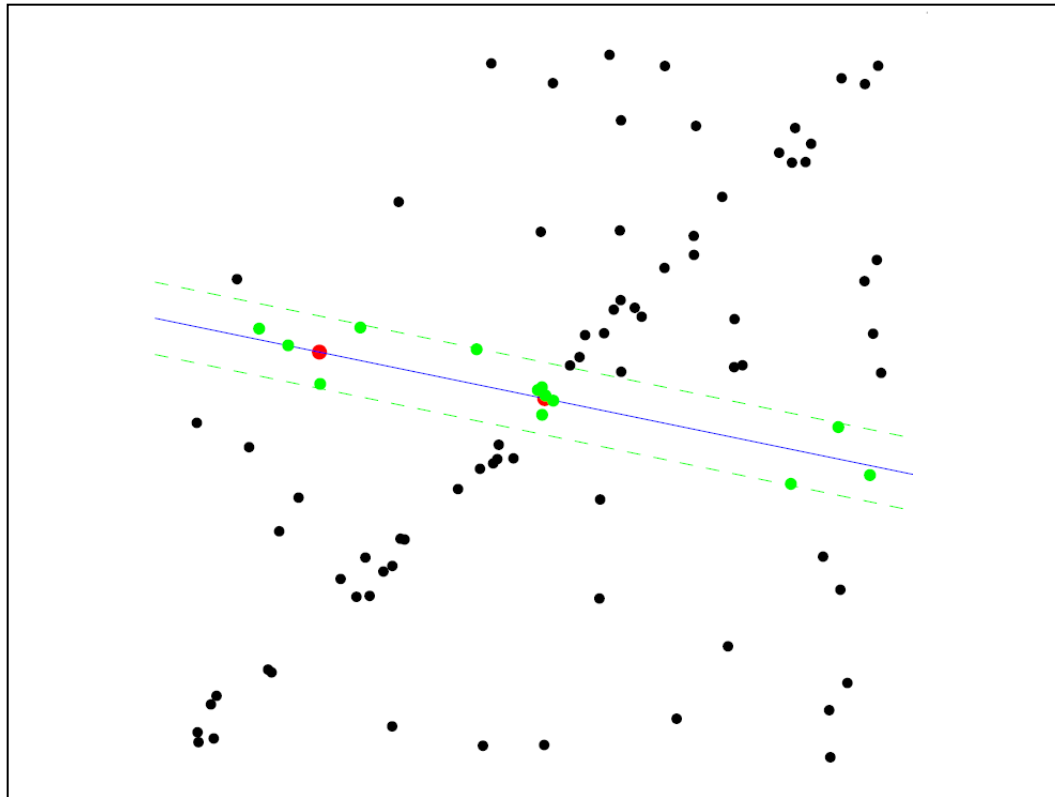
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. **Select points consistent with model**

RANSAC for line fitting example



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

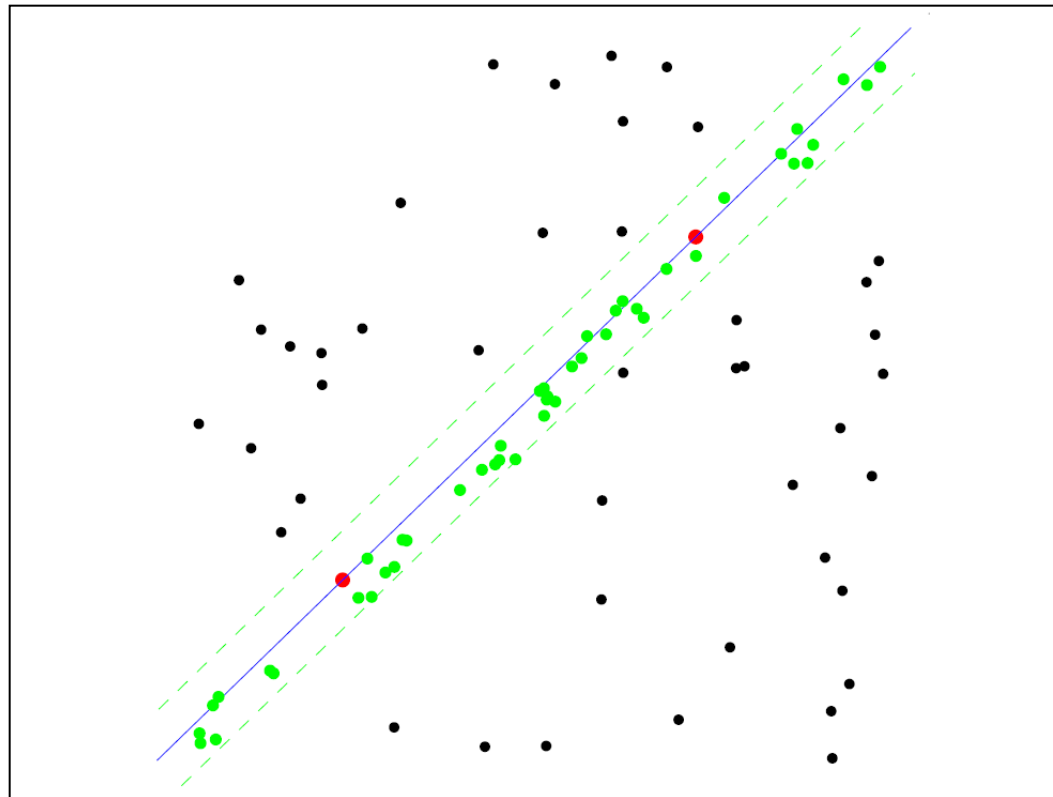
RANSAC for line fitting example



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

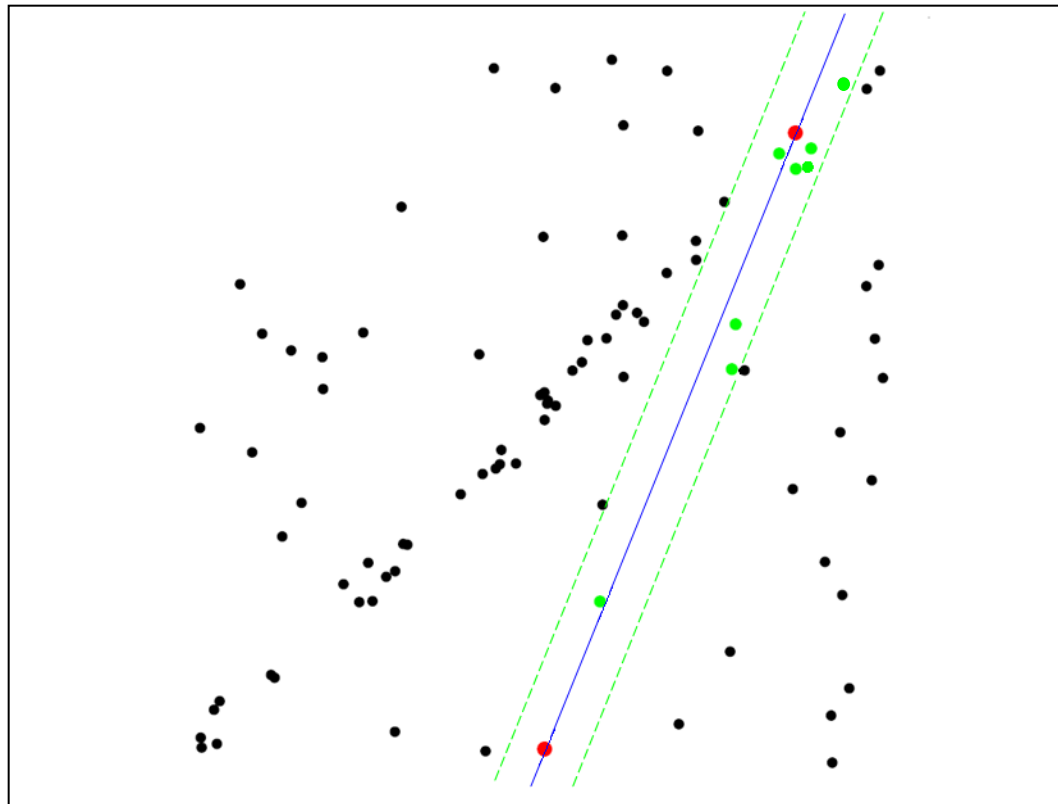
RANSAC for line fitting example

Uncontaminated sample



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC for line fitting example



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC for line fitting

- Repeat N times:
 - Draw s points uniformly at random
 - Fit a line (model) to these s points
 - Find *inliers* to this line among the remaining points (i.e., points whose distance from the line is less than t)
 - If there are d or more inliers, accept the line and refit using all inliers

Choosing the parameters

- Initial number of points s
 - Typically minimum number needed to fit the model
- Distance threshold t
 - Choose t so probability for inlier is p (e.g. 0.95)
 - Zero-mean Gaussian noise with std. dev. σ : $t^2=3.84\sigma^2$, $t=1.96\sigma$
- Number of samples N
 - Choose N so that, with probability p , at least one random sample is free from outliers (e.g. $p=0.99$) (outlier ratio: e)

Choosing the parameters

- Initial number of points ***s***
 - Typically minimum number needed to fit the model
- Distance threshold ***t***
 - Choose ***t*** so probability for inlier is p (e.g. 0.95)
 - Zero-mean Gaussian noise with std. dev. σ : $t^2=3.84\sigma^2$
- Number of samples ***N***
 - Choose ***N*** so that, with probability p , at least one random sample is free from outliers (e.g. $p=0.99$) (outlier ratio: e)

$$\left(1 - (1 - e)^s\right)^N = 1 - p$$

$$N = \log(1 - p) / \log(1 - (1 - e)^s)$$

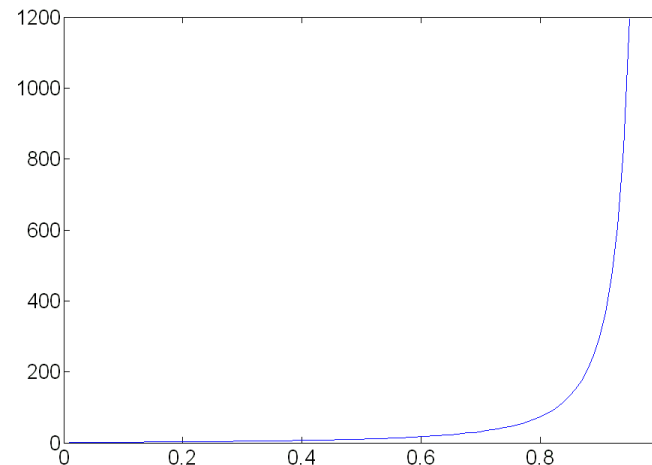
s	proportion of outliers e						
	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

Choosing the parameters

- Initial number of points ***s***
 - Typically minimum number needed to fit the model
- Distance threshold ***t***
 - Choose ***t*** so probability for inlier is p (e.g. 0.95)
 - Zero-mean Gaussian noise with std. dev. σ : $t^2 = 3.84\sigma^2$
- Number of samples ***N***
 - Choose ***N*** so that, with probability p , at least one random sample is free from outliers (e.g. $p=0.99$) (outlier ratio: e)

$$\left(1 - (1 - e)^s\right)^N = 1 - p$$

$$N = \log(1 - p) / \log(1 - (1 - e)^s)$$



Choosing the parameters

- Initial number of points **s**
 - Typically minimum number needed to fit the model
- Distance threshold **t**
 - Choose **t** so probability for inlier is p (e.g. 0.95)
 - Zero-mean Gaussian noise with std. dev. σ : $t^2 = 3.84\sigma^2$
- Number of samples **N**
 - Choose **N** so that, with probability p , at least one random sample is free from outliers (e.g. $p=0.99$) (outlier ratio: e)
- Consensus set size **d**
 - Should match expected inlier ratio

Adaptively determining the number of samples

- Outlier ratio e is often unknown a priori, so pick worst case, e.g. 50%, and adapt if more inliers are found, e.g. 80% would yield $e=0.2$
- Adaptive procedure:
 - $N = \infty$, *sample_count* = 0
 - While $N > \textit{sample_count}$
 - Choose a sample and count the number of inliers
 - If inlier ratio is highest of any found so far, set $e = 1 - (\text{number of inliers})/(\text{total number of points})$
 - Recompute N from e :

$$N = \log(1 - p) / \log(1 - (1 - e)^s)$$

- Increment the *sample_count* by 1

Template matching

Template matching - example



We want to localize
in the image this
template



template
one way traffic sign

- simplest method for object detection
- for every pixel in the image, compare the template with the window centered at that pixel: measure how well (in similarity or distance) the image window pattern resembles the template

Template matching - example



We want to localize in the image this template



- simplest method for object detection
- for every pixel in the image, compare the template with the window centered at that pixel: measure how well (in similarity or distance) the image window pattern resembles the template

Template matching - example

Similarity map



Highest value of the
similarity map



template

- simplest method for object detection
- for every pixel in the image, compare the template with the window centered at that pixel: measure how well (in similarity or distance) the image window pattern resembles the template

Template matching - example

Similarity map



Highest value of the
similarity map



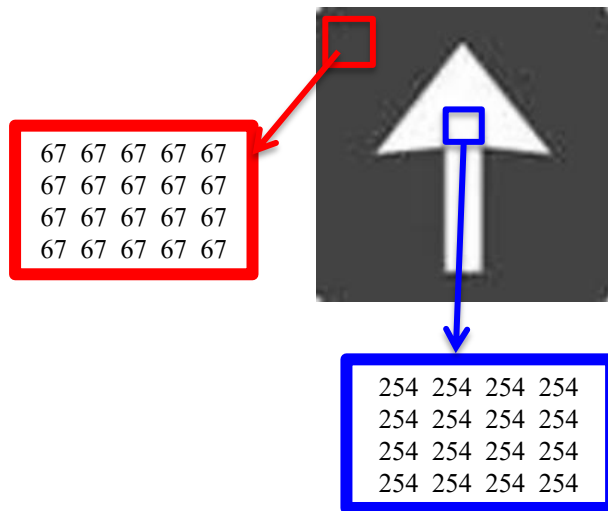
template

- similarity map = filtered image (template = filter)
- what is a good measure for similarity of two patches (windows)?
 - correlation?
 - sum of squared distances?
 - something else?

Template matching with filters

- template = 

rgb2gray
cv.cvtColor



rgb2gray = **cv.cvtColor**



Template matching with filters

- filter $f_1 = \uparrow$
- correlation: we filter image with filter f_1

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_1(u, v) I(i + u, j + v)$$



I

$$O_1 = f_1 \otimes I$$

What is the result of applying the filter?

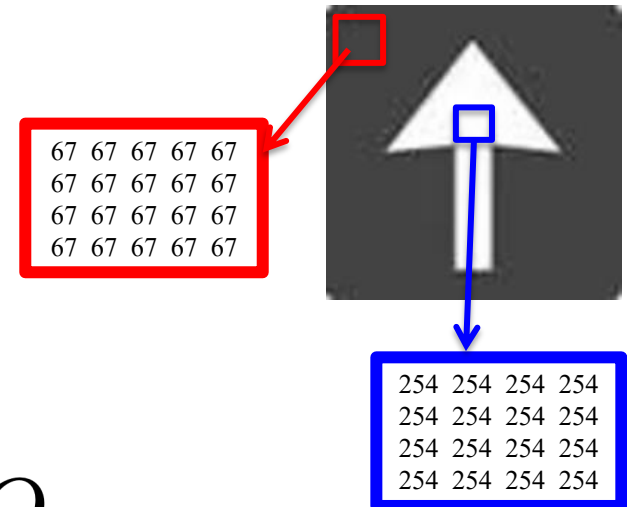
Template matching with filters

- filter $f_1 = \begin{array}{|c|} \hline \uparrow \\ \hline \end{array}$
- correlation: we filter image with filter f_1

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_1(u, v) I(i + u, j + v)$$

Explanations?

Unnormalized filter
(sum > 1) results in filtered
image being much brighter



Filtered image = completely white

O_1

Template matching with filters

- filter $f_2 = \uparrow$ normalized (sum = 1: 67 $\rightarrow$ 0.00007, 254 $\rightarrow$ 0.0003)
- correlation: filter image with filter f_2

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_2(u, v) I(i + u, j + v)$$



I

$$O_2 = f_2 \otimes I$$

What is the result of applying the filter?

Template matching with filters

- filter $f_2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ normalized (sum = 1: 67 $\rightarrow$ 0.00007, 254 $\rightarrow$ 0.00003)
- correlation: filter image with filter f_2

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_2(u, v) I(i + u, j + v)$$



Explanations?

The filter's response is large for high intensity values in the image. We obtain the maximum response for a white patch in the image.

O_2

Template matching with filters

- filter $f_3 = \blacksquare \uparrow$ normalized (sum = 0, $f_3 = f_2 - \overline{f_2}$)
- correlation: filter image with filter f_3

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_3(u, v) I(i + u, j + v)$$



I

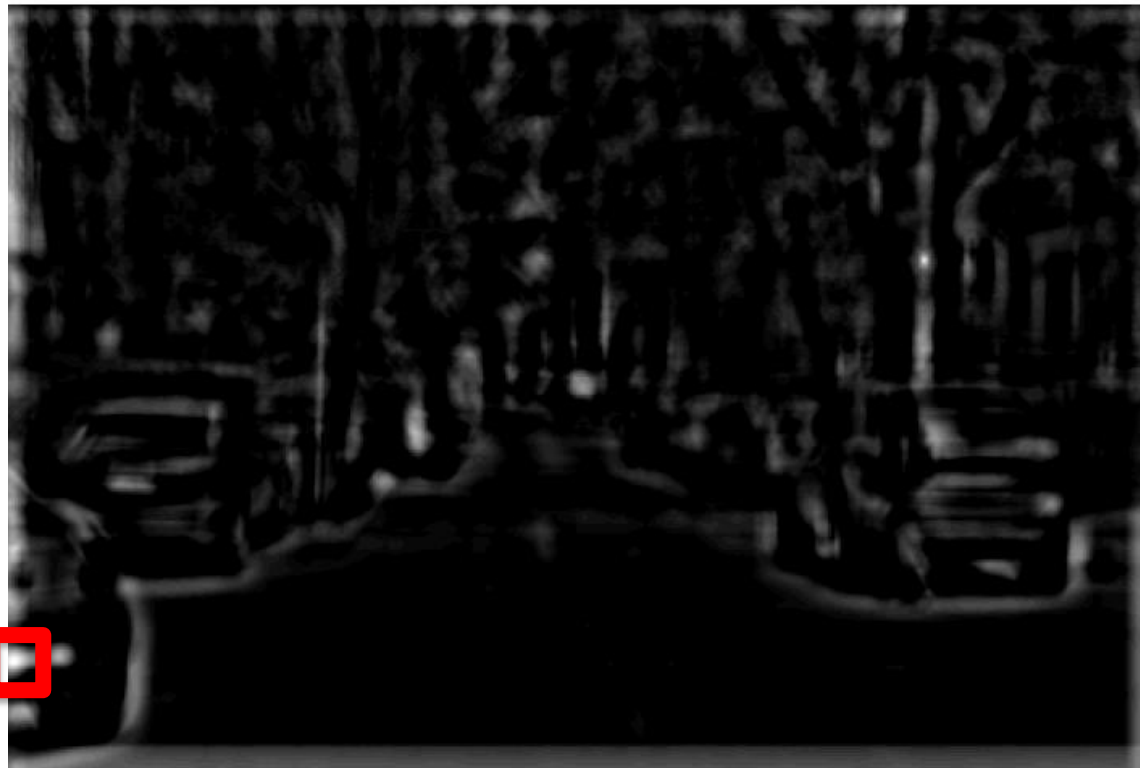
$$O_3 = f_3 \otimes I$$

What is the result of applying the filter?

Template matching with filters

- filter $f_3 = \blacksquare \uparrow$ normalized (sum = 0, $f_3 = f_2 - \overline{f_2}$)
- correlation: filter image with filter f_3

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_3(u, v) I(i + u, j + v)$$



Explanations?

Response is large when:

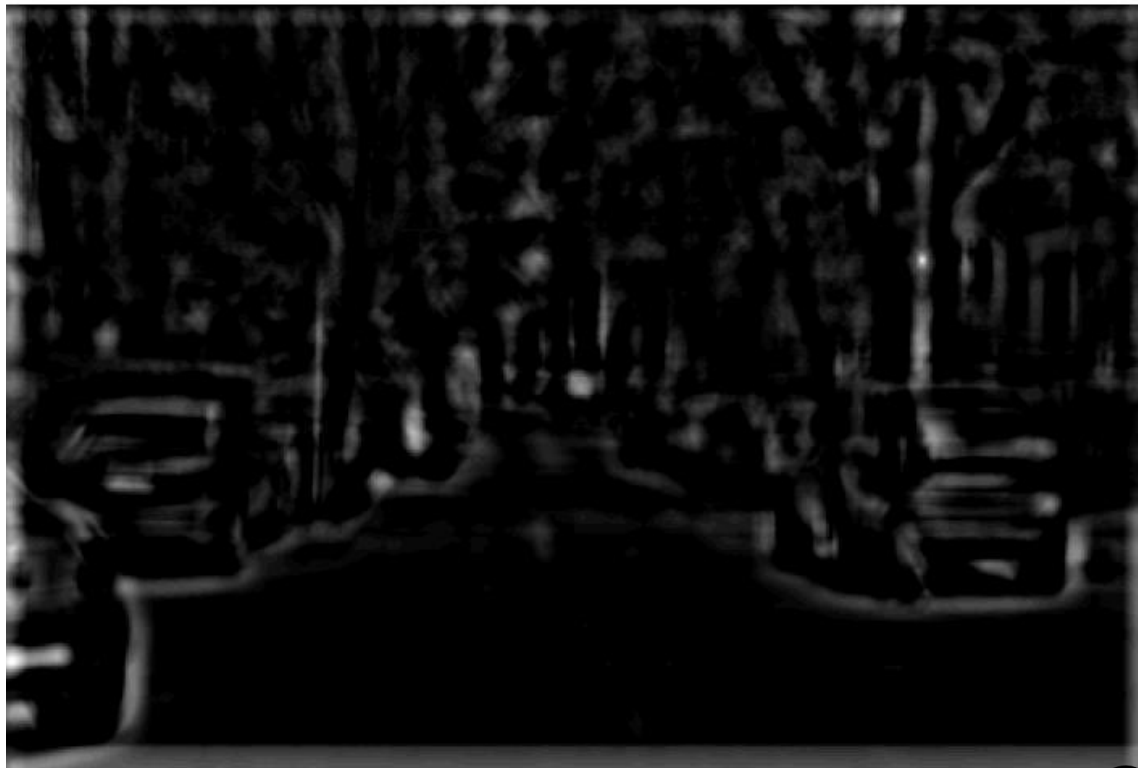
- small values (negative numbers) from the filter correlate with small pixel intensities in the image
- large values (positive numbers) from the filter correlate with large pixel intensities in the image

O_3

Template matching with filters

- filter $f_3 = \blacksquare \uparrow$ normalized (sum = 0, $f_3 = f_2 - \overline{f_2}$)
- correlation: filter image with filter f_3

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_3(u, v) I(i + u, j + v)$$



Explanations?

Response is maximum when:

- small values (negative numbers) from the filter correlate with black pixels
- large values (positive numbers) from the filter correlate with white pixels
- does not require pixel values in image to be near or proportional to values in filter.

O_3

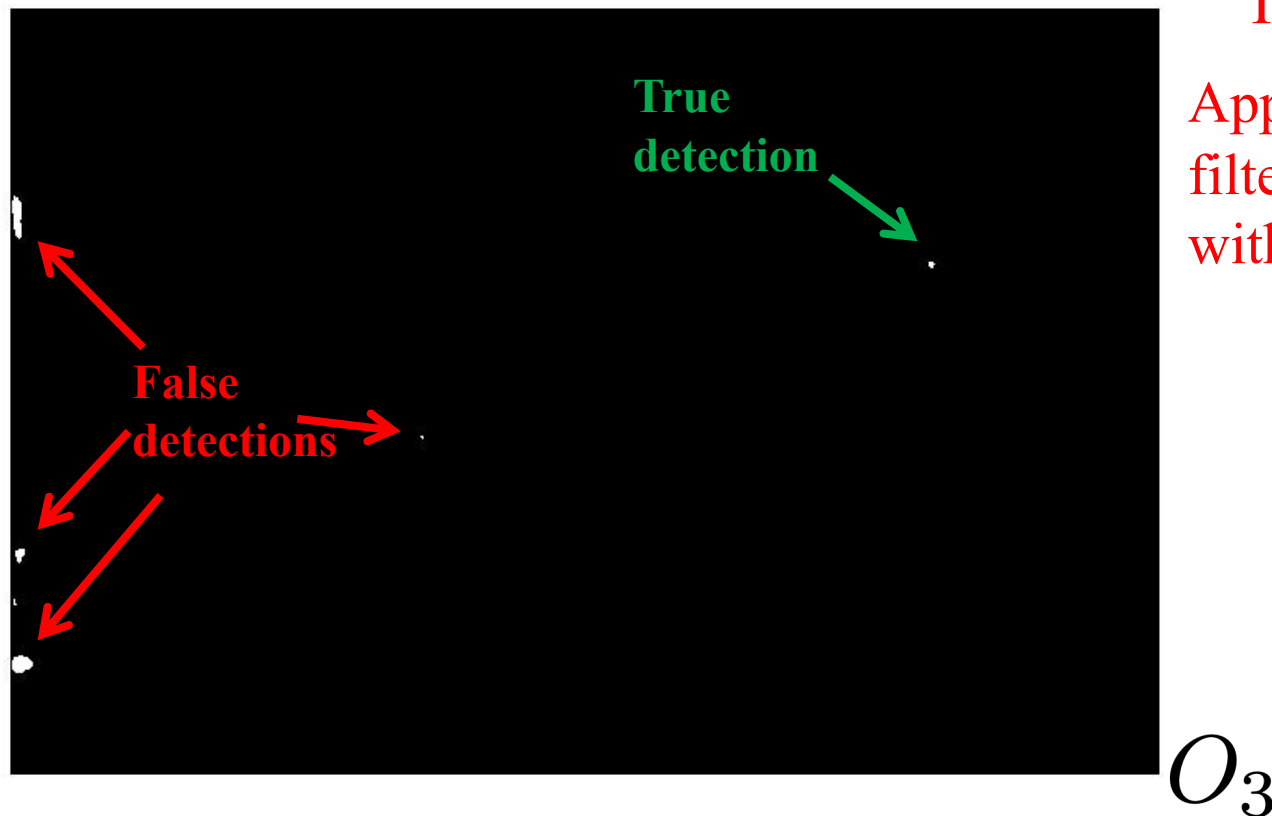
Template matching with filters

- filter $f_3 = \blacksquare \uparrow$ normalized (sum = 0, $f_3 = f_2 - \overline{f_2}$)
- correlation: filter image with filter f_3

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l f_3(u, v) I(i + u, j + v)$$

Thresholding

Apply a threshold to the filtered image: find all pixels with intensity $>$ threshold



Template matching with filters

- filter $f_1 = \uparrow$
- sum of squared distances

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - f_1(u, v))^2$$



I

Template matching with filters

- filter $f_1 = \uparrow$
- sum of squared distances

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - f_1(u, v))^2$$



Find the minimum value of distance / maximum value of similarity and this should correspond to the template.

$$O_4 (1 - \text{distance})$$

Template matching with filters

- filter $f_1 = \uparrow$
- sum of squared distances

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - f_1(u, v))^2$$



Shadow effects in the image will affect the template.

What to we obtain in this case?

Template matching with filters

- filter $f_1 = \blacksquare \uparrow$
- sum of squared distances

$$o(i, j) = \sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - f_1(u, v))^2$$



Explanations?

The filter's response is sensitive to the mean intensity in the window

$O_5 (1 - \text{distance})$

Template matching with filters

- filter $f_1 = \uparrow$

`res = cv.matchTemplate(img,template,method)`

- normalized correlation (of mean 0)

mean intensity in the
current window in
the image

mean intensity of
the filter

$$o(i, j) = \frac{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})(f_1 - \bar{f}_1)}{\sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})^2} \sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (f_1 - \bar{f}_1)^2}}$$

standard deviation of intensities in the current window in the image
standard deviation of intensities in the filter

**Cosine of two
normalized vectors**

Invariance to photometric changes

- filter $f_1 = \begin{array}{|c|} \hline \uparrow \\ \hline \end{array}$

`res = cv.matchTemplate(img,template,method)`

- normalized correlation

$$o(i, j) = \frac{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{u,v})(f_1(u, v) - \bar{f}_1)}{\sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{u,v})^2} \sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (f_1(u, v) - \bar{f}_1)^2}}$$

- brightness changes (brightness $\rightarrow b$):

$$I_{[u-k:u+k] \times [v-l:v+l]} = f_1 + b$$

- contrast changes (contrast $\rightarrow c$):

$$I_{[u-k:u+k] \times [v-l:v+l]} = c \times f_1$$

- brightness + contrast changes (affine transformation):

$$I_{[u-k:u+k] \times [v-l:v+l]} = b + c \times f_1$$

- in all cases $o(i, j)$ takes the same value = 1

Template matching with filters

- filter $f_1 = \uparrow$

`res = cv.matchTemplate(img,template,method)`

- normalized correlation

$$o(i, j) = \frac{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})(f_1 - \bar{f}_1)}{\sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})^2} \sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (f_1 - \bar{f}_1)^2}}$$



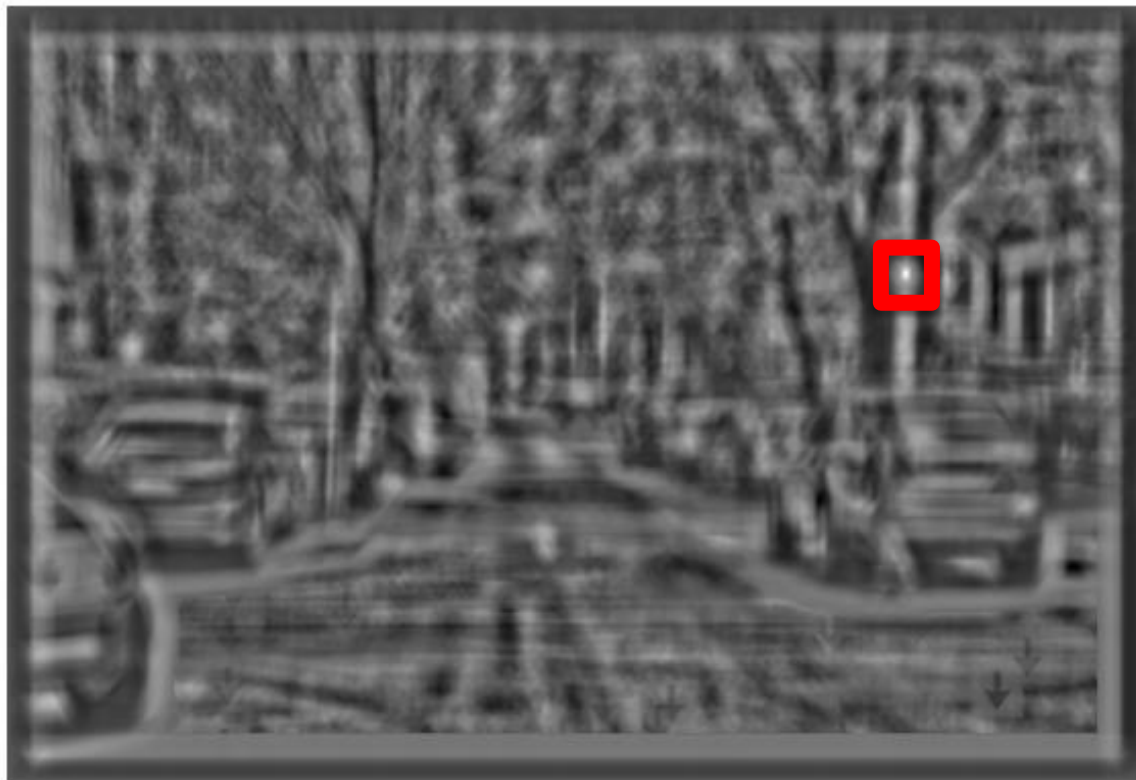
Template matching with filters

- filter $f_1 = \uparrow$

`res = cv.matchTemplate(img,template,method)`

- normalized correlation

$$o(i, j) = \frac{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})(f_1 - \bar{f}_1)}{\sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})^2} \sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (f_1 - \bar{f}_1)^2}}$$



finding the minimum value =
detect the template

Template matching with filters

- filter $f_1 =$

`res = cv.matchTemplate(img,template,method)`

- normalized correlation

$$o(i, j) = \frac{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})(f_1 - \bar{f}_1)}{\sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})^2} \sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (f_1 - \bar{f}_1)^2}}$$



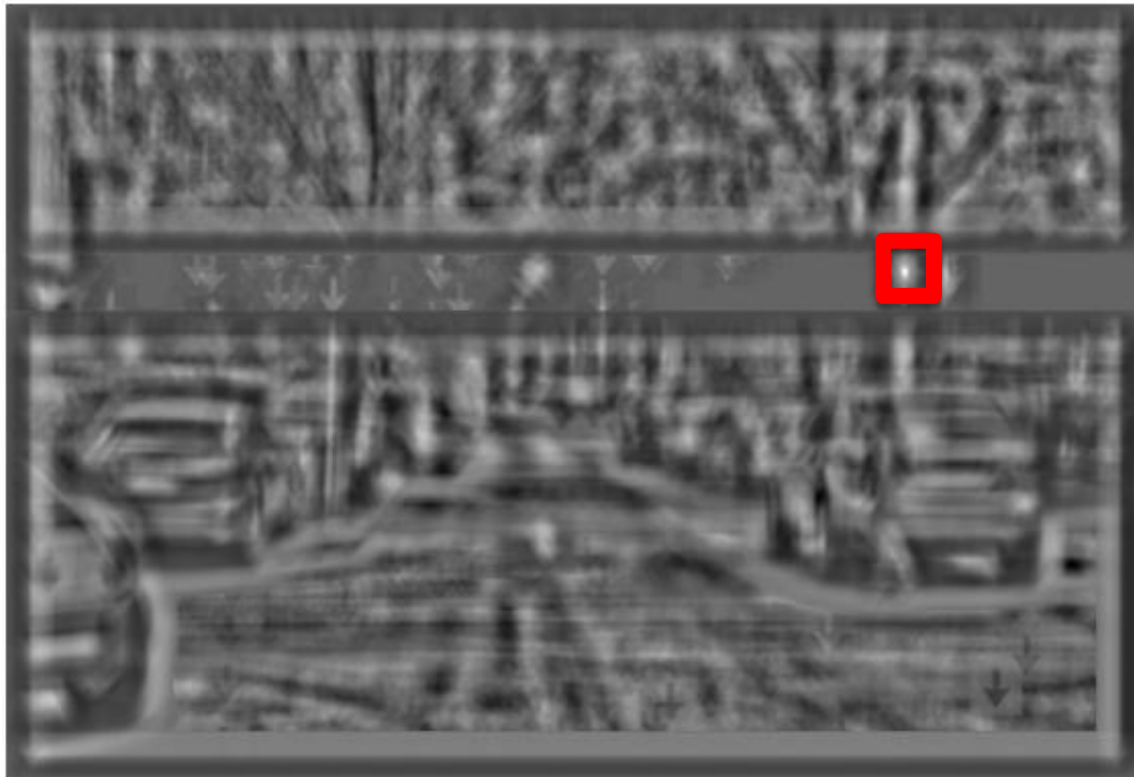
Template matching with filters

- filter $f_1 = \uparrow$

`res = cv.matchTemplate(img,template,method)`

- normalized correlation

$$o(i, j) = \frac{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})(f_1 - \bar{f}_1)}{\sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (I(i+u, j+v) - \bar{I}_{i,j})^2} \sqrt{\sum_{u=-k}^k \sum_{v=-l}^l (f_1 - \bar{f}_1)^2}}$$



finding the minimum value =
detect the template

Waldo



Where's Waldo?

Waldo



Best measure of similarity?

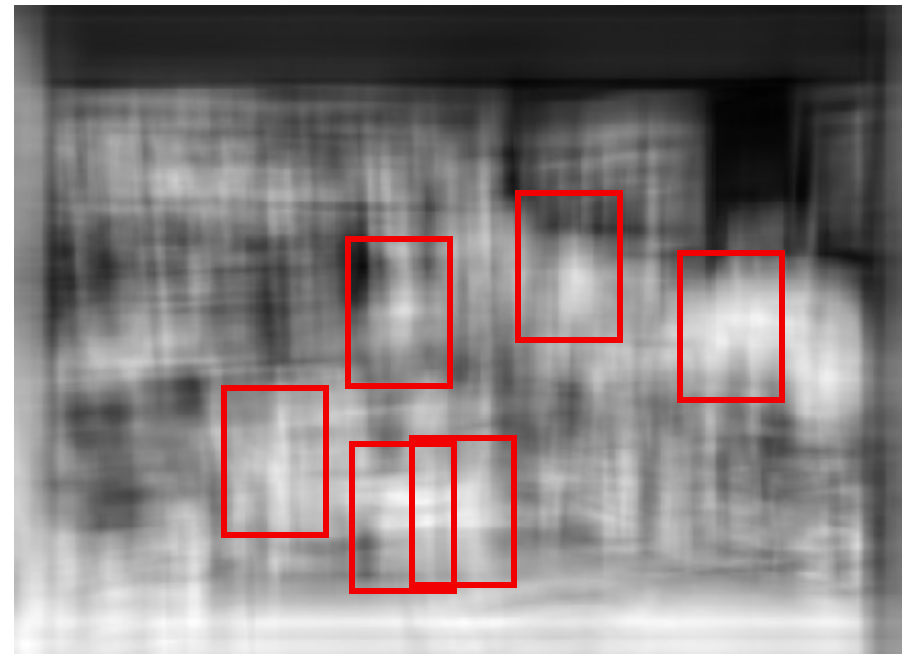
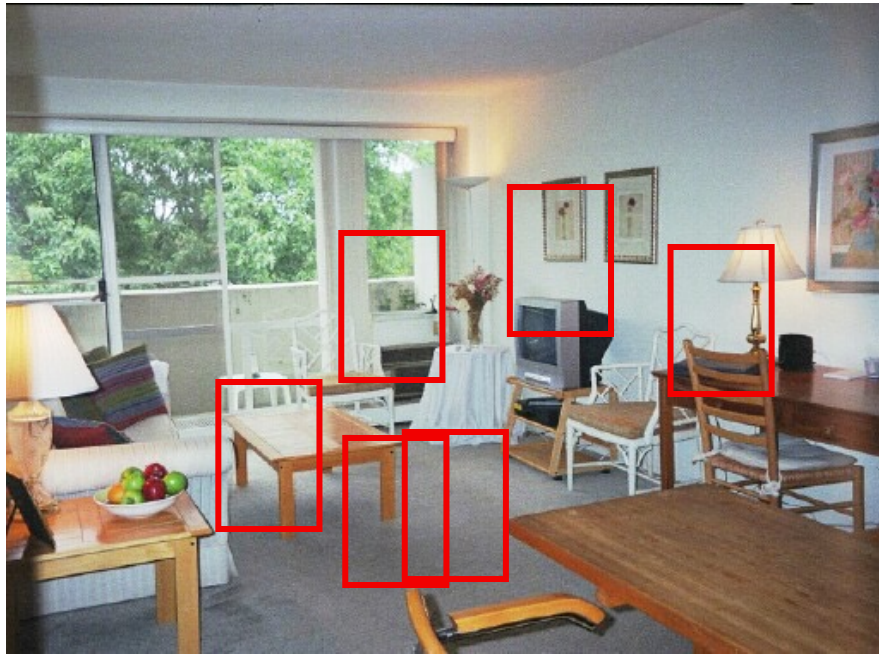
- correlation with filter of mean 0: results are not so good (false detections)
- sum of squared distances: sensitive to mean intensity
- normalized correlation: invariant to mean intensity and contrast





Object detection with template matching

Detect chairs in the image



Template matching doesn't solve the problem

A “popular method is that of template matching, by point to point correlation of a model pattern with the image pattern. These techniques are inadequate for three-dimensional scene analysis for many reasons, such as occlusion, changes in viewing angle, and articulation of parts.” Nevatia & Binford, 1977.

Object detection with template matching

Pros:

- simple method, easy to implement
- good results also in cases when the template/ something very similar to the template is in the image (some invariance to occlusions)

Cons:

- not invariant to scale, rotation (of the same template): if we resize the template or rotate it we obtain different results
- for object classes with large variability in appearance (cars, persons, faces, etc) we cannot apply template matching
- should work fine for logo detection, etc

Finding smaller or larger template

initial template 

smaller template   alternative build a pyramid with resized images



Texture